



Self-supervised domain adaptation for machinery remaining useful life prediction

Quy Le Xuan^{*}, Marco Munderloh, Jörn Ostermann

Institute for Information Processing - L3S Research Center, Leibniz Universität Hannover, Appelstr. 9A, 30167 Hannover, Germany

ARTICLE INFO

Keywords:

RUL prediction
Predictive maintenance
Prognostics and health management
Domain adaptation
Self-supervised learning
Siamese neural network

ABSTRACT

Remaining useful life (RUL) prediction presents one of the most crucial tasks in modern machinery prognostics and health management systems. As a powerful data-driven solution, deep learning has shown its promising potential in accurately predicting the RUL based on historical condition monitoring data. However, deep learning-based methods typically require the training and test data to be drawn from the same distribution or domain, which is usually not the case in real-world application scenarios. Unsupervised domain adaptation (UDA) methods have been proposed to address this domain shift problem, but most of them focus only on learning domain-invariant feature representations while forcing the prediction error to be low on the source labeled data. Empirical observations have shown that this kind of domain adaptation is insufficient to guarantee good generalization in the target domain. To overcome this limitation, we propose a novel self-supervised domain adaptation (SSDA) framework that additionally incorporates the intrinsic information of the target domain data into the domain adaptation process without the need for its RUL labels. We developed a dual Siamese network-based training pipeline that enables the optimization for the self-supervised task in both the source and target domains to be realized jointly in conjunction with the base UDA objectives. Evaluation results from extensive experiments on the benchmark C-MAPSS dataset of aircraft turbofan engines show the superiority of our proposed framework over other state-of-the-art methods. On average, we achieve an improvement of 20.1% and 51.2% on two different performance metrics.

1. Introduction

Prognostics and health management (PHM) systems, as a key enabler for developing advanced predictive maintenance strategies, are typically constructed by incorporating interdisciplinary technologies with the goal of not only monitoring and assessing the current health state but also predicting the remaining useful life (RUL) of the systems of interest [1]. With the ability to reliably predict when failures are likely to occur, maintenance can be efficiently scheduled and performed in a proactive manner. This can make a significant contribution to avoiding unplanned downtime, reducing maintenance costs, and improving equipment availability and effectiveness. Traditionally, RUL prediction can be realized by means of model-based approaches [2,3]. In principle, model-based methods typically aim to build mathematical models representing the degradation process of the system of interest based on assumptions about the underlying deterioration mechanisms. Despite the benefits of the model interpretability, the requirement of prior expert knowledge about the degradation mechanisms and the challenges in modeling them mathematically with an appropriate level

of detail are the main factors that hinder the application of model-based methods in practice, especially in cases of complex systems [4]. With the increased availability of condition monitoring data and the recent significant advances in machine learning techniques, in particular deep learning (DL), data-driven methods have become more prevalent for solving the RUL prediction task [5]. In contrast to conventional machine learning approaches that often require domain-specific expert knowledge for manual feature engineering, DL is capable of automatically learning representative features from the raw sensor readings. Recent years have seen an increasing interest from both academic and industrial communities in developing various DL-based methods for the task of RUL prediction [6–14]. In principle, the superior performance of DL-based models can be ensured only if they are trained on a sufficiently large amount of labeled data, while the training and test data are expected to be drawn from the same data-generating distribution. However, acquiring labeled data for the task of RUL prediction, i.e. the run-to-failure data, is inherently an expensive and time-consuming process. Thus, in most real-world application scenarios, only a limited amount of labeled data is available for training, and the test data

^{*} Corresponding author.

E-mail address: lexuan@tnt.uni-hannover.de (Q. Le Xuan).

can be out-of-distribution compared to the training data due to the diversity in operating conditions and underlying fault modes [15,16]. The difference in the characteristics as well as the distributions of the training and test data is referred to as the distribution or domain shift. To address this domain shift problem, unsupervised domain adaptation (UDA) methods [17] have been proposed, but most of them focus only on learning feature representations that are transferable between the domains by aligning their marginal distributions and minimizing the prediction error on the labeled source data. However, our empirical observations have shown that domain-invariant feature representations and low prediction error in the source domain are insufficient to guarantee good generalization in the target domain. Motivated by these empirical observations, in this work we propose a novel self-supervised domain adaptation (SSDA) framework for the task of RUL prediction that additionally incorporates the intrinsic information of the target-domain data into the domain adaptation process in a self-supervised manner, without the target labels having to be known. Throughout this paper, the term target labels will be used for short to refer to the actual RUL labels in the target domain, where the explicit use of the full term is not necessary. Our main contributions are summarized as follows:

- To overcome the limitation of the existing UDA methods, we propose a novel domain adaptation framework for the task of RUL prediction that additionally incorporates the intrinsic information of the target-domain data into the domain adaptation process in a self-supervised manner. Our self-supervised pre-text task is specifically designed for the RUL prediction use case.
- We propose a novel dual Siamese network-based training pipeline that enables the integration of the additional self-supervised learning part into the existing UDA frameworks (discrepancy-based and adversarial).
- We evaluated the performance of our proposed framework by conducting extensive experiments while deploying advanced Bayesian hyperparameter optimizations for 12 domain adaptation scenarios, each with two different pre-processing pipelines, on the benchmark C-MAPSS dataset of aircraft turbofan engines. The experimental results showed the superior performance of our proposed method over other state-of-the-art methods in almost all evaluation scenarios.

The remainder of this paper is structured as follows. Section 2 gives an overview of existing works most related to our present work. In Section 3, we provide the background as well as the context of the domain adaption for the task of RUL prediction. The concepts and details of our proposed SSDA framework are presented in Section 4. In Section 5, we first describe the experimental setups and implementation details, then report and discuss the evaluation results of our proposed methods in comparison with other state-of-the-art methods as well as with other baselines in the form of ablation studies. The paper ends with conclusions in Section 6.

2. Related work

Inspired by the remarkable advances in various fields such as computer vision and natural language processing in recent years, DL has been gaining increasing attention as an efficient solution for the task of RUL prediction. A large number of deep neural network architectures have been proposed towards solving this challenging task, including, for example, convolutional neural networks (CNNs) [6,7], long short-term memory (LSTM) [8,9], CNN-LSTM [10], autoencoder [11, 12], and transformer [13,14]. The key characteristic that enables the superior performance of DL-based methods is their capability of automatically extracting relevant features capturing complex non-linear relationships between inputs and outputs while mitigating or even bypassing the need for manual feature engineering. As a supervised data-driven method, DL models have to be trained on a large amount of labeled data. Furthermore, the test data, on which the DL models

should work and make predictions, is assumed to share similar characteristics as those of the training data. In other words, the training and test data are expected to be in-distribution. However, acquiring labeled data for the RUL prediction task, i.e. run-to-failure data, is extremely expensive and time-consuming. Therefore, in practice it is often the case that only a limited amount of labeled training data is available and the test data can be out-of-distribution, i.e. its characteristics are different from those of the training data. As a result, DL models which are trained in a conventional supervised manner on a limited amount of labeled training data often show poor generalization performance when being applied to test data in real-world application scenarios [15].

UDA techniques [18–29] have been developed to address the domain shift problem between the training and test data and have shown significant progress in various application fields. Most of the existing UDA methods share the common base concept: training models for the task at hand using the labeled source data while aligning the distributions of the source and target data at the input or feature level, such that the trained model (with or without further fine-tuning) can generalize well on the unlabeled target domain data. For example, CORrelation ALIGNment (CORAL) [22] aims to minimize the domain gap between the source and target domains by aligning the second-order statistics of their features. Similarly, [20,21,23] proposed to learn domain-invariant features by fooling a domain discriminator using adversarial training while minimizing the empirical source risk. Previous research on domain adaptation has mainly focused on classification tasks [17]. In the field of PHM, UDA methods have thus been widely adopted and further developed for fault diagnostics [30,31]. Recently, a growing number of UDA-based methods have also been proposed for RUL prediction [32–39]. As one of the first notable works addressing the domain shift problem for RUL prediction, da Costa et al. [32] proposed a deep UDA approach based on the adversarial domain alignment technique [20,21] while using LSTM as a feature extractor. This work aimed to align the source and target data at the feature level by training the feature extractor and a domain discriminator in an adversarial manner with the help of a reversal gradient layer [20,21]. At the same time, a further regressor composed of fully-connected layers was jointly trained using the labeled source data to ensure that the learned features are representative for the underlying task. The backbone predictive model composed of the feature extractor and the regressor was then directly used to make predictions for the target domain. Similarly, [33] proposed another deep UDA method for RUL prediction called ADARUL. It is built on adversarial discriminative domain adaptation (ADDA) introduced in [23] where a multi-layer bidirectional LSTM network was employed as the feature extractor. ADARUL [33] enabled the learning of the domain-invariant features by fooling the domain discriminator using GAN loss function with inverted labels [23,40]. Furthermore, to confuse the domain discriminator, the target feature extractor was further fine-tuned while using the source feature extractor as an initialization. With the same goal of reducing the distribution shift between the learned features of the source and target domains, Fu et al. [34] simultaneously integrated multi-kernel maximum mean discrepancies and adversarial training into their UDA framework, where a deep residual LSTM served as the feature extractor. Hu et al. [37] presented a Wasserstein distance-based weighted domain adversarial neural network for RUL prediction under different working conditions with two domain classifiers taking into account the similarity of source and target samples.

Although these kinds of UDA methods have shown to be capable of enhancing the generalization for the target domain in some scenarios where the magnitude of the domain shift is relatively small, they often suffer from the negative transfer or the mode collapse, especially when the domain shift is more significant. The reason for this is that these UDA methods only focus on aligning the marginal feature distributions while keeping the source risk low. Our empirical observations have shown that even if the marginal feature distributions are perfectly aligned, the features of the source and target domains can still be

semantically misaligned. This will be the case, for example, when the feature vector of a source data sample with a small label is misaligned to be located close to the feature vector of another target data sample with a large label. Thus, there is a need to incorporate the intrinsic information of the target domain data when designing domain adaptation algorithms based on learning invariant representations [41]. To overcome this limitation, we propose a novel SSDA framework that further regularizes the learned features by solving a common pre-text task in both the source and target domains without the need for using ground-truth RUL labels. Sharing the similar idea of leveraging the target-specific information for the domain adaptation with our work, Ragad et al. [35] proposed CADA that learns domain-invariant features while further maximizing the mutual information between the target input data and the target latent features using InfoNCE loss [42,43]. Unlike [35], in this work we aim to additionally align the source and target features in such a way that the deviation of each predicted RUL pair matches the temporal distance of the corresponding input sample pair. Zhang et al. [36] explicitly regularized the features of the source and target domains using predefined sets of constraints based on assumptions about the properties of the features in the healthy state as well as during the degradation process. In contrast, the source and target features learned by our method are implicitly semantically aligned by exploiting the inherent nature of the RUL prediction task without having to make assumptions about the feature properties. Zhang et al. [44] proposed to improve the generalization of the RUL prediction for aero-engines under working condition shifts by training a transformer-based model with multi-task learning in the absence of the target domain data. Two supervised auxiliary tasks were introduced based on assumptions about the health state and the degradation behavior.

Self-supervised learning (SSL) [45] is a powerful learning paradigm that aims to extract useful feature representations on the basis of the inherent structure and pattern of the available data without requiring human annotations or labeled data. SSL-based methods have been leveraged successfully to improve the learning performance across a wide range of application areas and data modalities [42,45,46]. Recent years have also seen an increasing number of generative SSL-based approaches developed for the task of RUL prediction, e.g. [47–49]. Guo et al. [47] proposed to utilize unlabeled data via self-supervision in a pre-training phase using a masked autoencoder with transformers and gated convolutional units being the basic components of the network architecture. The self-supervision task for the masked autoencoder was to reconstruct the masked patches of an input sample given its unmasked patches. Akrim et al. [48] investigated and compared the benefit of pre-training a gated recurrent unit-based encoder within an autoencoder and an autoregressive model on the available unlabeled data in a self-supervised manner. The pre-trained encoder was then used to initiate the feature extractor in the self-supervised fine-tuning phase for the RUL prediction. Instead of pre-training a feature extractor for a pre-text task with pseudo-labels as in [47,48], Ding et al. [49] introduced an adversarial out-domain augmentation (AOA) framework that generates pseudo-domains to increase the diversity of data available for training. Although the difference between the pseudo-domains and the source domains is desirable for data diversity, the authors proposed to add semantic and manifold regularization to the objective function to promote domain consistency as well as to avoid excessive differences. Similar to our work, several other studies that also follow the concept of using SSL and Siamese networks [50] to improve the performance of RUL predictive models were recently proposed [51,52]. However, these works did not deal with the domain adaptation problem and their pre-text tasks also differ from that proposed in our work. Krokotsch et al. [51] defined their pre-text task to align the distance of the extracted features in the latent space with the deviation of the corresponding RUL labels in the output space, while Vermelin et al. [52] proposed learning representative features to distinguish between correctly ordered and shuffled time series sub-sequences. Both

of these works used the feature extractor pre-trained for their pre-text task to train a downstream predictor for the actual RUL prediction task afterwards in a supervised manner using labeled data. In contrast, we propose SSDA that applies SSL in conjunction with the UDA concept to improve cross-domain generalization without the need for target-domain labels. Instead of using a pair of identical sub-networks as in the case of conventional Siamese neural networks, we propose a framework with two pairs of identical backbone networks working in tandem, each dedicated for one domain. Our novel dual Siamese neural network-based training pipeline along with the task-specific self-supervision was designed in such a way that allows the additional SSL part to be integrated into the existing UDA frameworks (discrepancy-based and adversarial), resulting in a more robust framework for the task of domain adaptive RUL prediction.

CADA [35] and AOA [49] can be interpreted as belonging to the family of self-supervised domain adaptation and generalization methods, although they are not explicitly referred to as such in the articles that originally proposed them. The main advantage of CADA lies in its ability to incorporate target-specific information into the domain adaptation process to improve cross-domain RUL prediction (similar to our framework), while AOA deals with a more challenging scenario where no data of the target domain is available. The main drawback of these methods lies in the fact that their proposed self-supervision is not explicitly tailored to the RUL prediction task, but rather task-agnostic, omitting the task-specific characteristics reflected in the temporally evolving nature of the degradation data. In contrast, the self-supervised pre-text task of our SSDA is specifically designed for the RUL prediction use case, taking into account the degradation nature of the data.

3. Background

Consider a learning scenario where we have two datasets drawn from two related but different domains, namely source domain $\mathcal{S}$ and target domain $\mathcal{T}$. Let $\mathcal{X}$ and $\mathcal{Y}$ be the input and output space shared by these two domains. The source-domain dataset $\mathcal{D}_S$ is labeled, containing N_S input-label pairs:

$$\mathcal{D}_S = \left\{ \left(\mathbf{X}_S^{(n)}, \mathbf{y}_S^{(n)} \right) \mid \mathbf{X}_S^{(n)} \in \mathcal{X}, \mathbf{y}_S^{(n)} \in \mathcal{Y} \right\}_{n=1}^{N_S}. \quad (1)$$

In contrast, the target-domain dataset $\mathcal{D}_T$ is unlabeled, i.e. the input data is available but its labels (aka ground-truth outputs) are unknown:

$$\mathcal{D}_T = \left\{ \mathbf{X}_T^{(n)} \mid \mathbf{X}_T^{(n)} \in \mathcal{X} \right\}_{n=1}^{N_T}. \quad (2)$$

Each domain is characterized by a marginal distribution $p_{\mathcal{Y}}(X)$ from which the input data are drawn and a mapping function $f_{\mathcal{Y}} : \mathcal{X} \rightarrow \mathcal{Y}$ that represents the actual input-label relationship [41,53,54]. Here, $\mathcal{Y}$ can be replaced by $\mathcal{S}$ and $\mathcal{T}$ for the source and the target domain, respectively. Given a labeled source-domain dataset and an unlabeled target-domain dataset as described above, the goal is to train a model that can generalize well for the target-domain data. Note that owing to the absence of labels on the target-domain dataset, it is impossible to apply supervised learning for the target domain. Moreover, due to the domain shift, models trained in a standard supervised manner on the source domain often suffer from poor performance on the target domain.

In the context of RUL prediction, an input sample $\mathbf{X}^{(n)}$ is typically a multivariate time series containing condition monitoring data of a system of interest collected by M sensors for the last L time steps including the current one. Mathematically, the n th input sample can be formulated as follows:

$$\mathbf{X}^{(n)} = \left(\mathbf{x}_1^{(n)}, \mathbf{x}_2^{(n)}, \dots, \mathbf{x}_M^{(n)} \right) \in \mathbb{R}^{L \times M}, \quad (3)$$

where

$$\mathbf{x}_m^{(n)} = \left(x_{m,1}^{(n)}, x_{m,2}^{(n)}, \dots, x_{m,L}^{(n)} \right)^T \in \mathbb{R}^L, \quad (4)$$

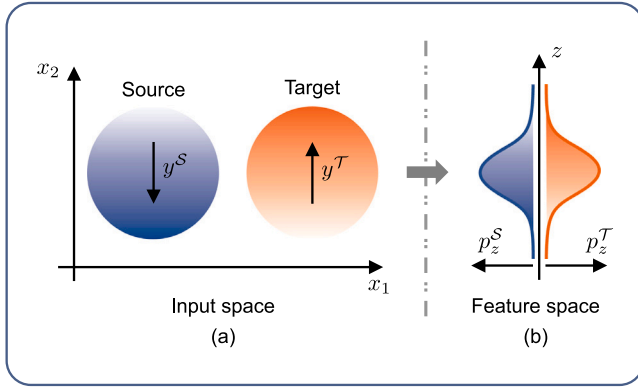


Fig. 1. A counterexample for a regression task where methods based only on domain-invariant features fail.

with $m \in \{1, 2, \dots, M\}$. Each label $y^{(n)}$ is a scalar representing the actual RUL of the system of interest computed from the current time step, i.e. the last time step of the corresponding input data sample $X^{(n)}$.

Like many other learning tasks, domain shift between the training and test data poses a significant challenge for the deployment of data-driven RUL predictive models in real-world application scenarios. In principle, there are multiple factors that can cause the domain shift between the training and test data, for example, the diversity of working conditions, the difference in underlying fault modes, as well as the production variability. These factors can lead not only to covariate shift [17,55], i.e. the shift of the input distribution, but also to concept shift [17,55], resulting in changes in the conditional distribution $p(y|X)$ as well as the input-output mapping function f .

For application scenarios where it can be assumed that the training and test data come from the same distribution, RUL prediction can be interpreted as a supervised regression task. In this case, RUL predictive models can be trained using a supervised loss, e.g. root mean square error (RMSE):

$$\mathcal{L}_{\text{RMSE}} = \frac{1}{N} \sum_{n=1}^N \|\hat{y}^{(n)} - y^{(n)}\|_2, \quad (5)$$

where $\hat{y}^{(n)}$ and $y^{(n)}$ are the n th predicted and ground-truth RUL, respectively.

4. Proposed method

Given source labeled condition-monitoring data $\mathcal{D}_S$ and target unlabeled data $\mathcal{D}_T$ as described in Section 3, our main goal is to learn a target model composed of a target feature extractor G_f^T and a target regressor G_r^T that can correctly predict the RUL on the target domain. In other words, we aim at training an RUL predictive model $G_f^T \circ G_r^T$ with a low target risk:

$$\mathcal{R}^T(G_f^T \circ G_r^T) = \mathbb{E}_{(X, y) \sim p^T} [\mathcal{L}_{\text{RMSE}}(G_f^T(G_r^T(X)), y)], \quad (6)$$

despite the absence of target labels. Here, feature extractor $G_f^T: \mathcal{X} \rightarrow \mathcal{Z}$ is responsible to learn discriminative features from the input by mapping it into a latent feature vector of a pre-defined dimension. Based on this extracted feature vector, the RUL regressor $G_r^T: \mathcal{Z} \rightarrow \mathcal{Y}$ will predict the corresponding RUL. As mentioned above, due to the absence of target labels, direct supervised training on the target domain is impossible. Therefore, to address this problem, UDA is typically applied instead. In general, UDA methods can be categorized into two main groups, namely generative and discriminative methods [23]. Our proposed method is built upon the latter one, i.e. discriminative UDA. The key idea of discriminative UDA is to learn a feature extractor G_f^S along with a task-specific predictor G_r^S on the source domain, which

can then be used for the target domain, either directly or with small adjustments. While the predictor G_r^S is typically kept unchanged for use in the target domain, i.e. $G_r^T = G_r^S$, the source feature extractor G_f^S can be either (1) directly used for the target domain, i.e. $G_f^T = G_f^S$, or (2) used as an initialization for the target feature extractor that will be further fine-tuned, i.e. $G_f^T \neq G_f^S$. In the following, we use G_f to refer to the unified notion of the feature extractor with $G_f(X) = G_f^S(X)$ if X is an input from the source domain, and $G_f(X) = G_f^T(X)$ if X comes from the target domain. In order to directly use the source predictor as a target predictor, most of the existing discriminative UDA methods aim to keep the learned features transferable across both domains using adversarial training as well as distribution discrepancy-based losses. Despite having been shown to be effective for some use cases [21,23,32], the main drawback of these methods is that they only align the marginal feature distributions while forcing the source domain error to be low using the available source labels. Previous works [41,56,57] proved that aligned feature representations and a low source risk are not sufficient to guarantee good generalization on the target domain, even if the marginal feature distributions of the source and target domains are perfectly aligned. In fact, this can happen due to misalignment of the conditional distribution of output given input features, $\bar{p}^S(y|G_f^S(X)) \neq \bar{p}^T(y|G_f^T(X))$. See Fig. 1 for a counterexample of a regression task where standard discriminative UDA methods fail, even if the marginal feature distributions are perfectly aligned and the source empirical error is minimized. (a) In the input space, the input data $\mathbf{x} = (x_1, x_2)^T$ of the source (blue) and target (orange) domain suffers from a domain shift w.r.t. x_1 -component. The ground-truth data-generating functions of both domains are defined as follows: $y^S(\mathbf{x}) = -w x_2$ and $y^T(\mathbf{x}) = w x_2$, where $w \in \mathbb{R}^+$. Here, the ground-truth values are encoded by the color intensity. (b) A regression model $G_f \circ G_r$ composed of a linear transformation $z = G_f(\mathbf{x}) = x_2$ and a downstream linear regressor $G_r(z) = -w z$ optimally fulfill the required properties of standard UDA methods: (i) minimizing the marginal distribution of the source and target features, and (ii) minimizing the empirical risk in the source domain. However, the misalignment of the conditional distributions $p^S(y^S|z)$ and $p^T(y^T|z)$ leads to poor performance of $G_f \circ G_r$ in the target domain: $G_r(G_f(\mathbf{x})) = -w x_2 \neq w x_2 = y^T(\mathbf{x})$.

Motivated by these observations, we develop a SSDA approach that enables the feature representations to be further aligned semantically in a self-supervised manner through solving a shared pre-text task in both the source and target domains. A schematic illustration of our proposed SSDA framework is depicted in Fig. 2. Our framework has a dual Siamese network-based architecture, composed of two pairs of identical backbone networks working in tandem. Each pair of the backbone networks ($G_f^V \circ G_r^V, G_f^V \circ G_r^V$) is responsible to process data of one domain, with $V = S$ for source domain and $V = T$ for target domain. Each backbone network is composed of an encoder G_f^V and a regressor G_r^V . In each forward pass during training, for each domain, two input samples, $u_i^V X(t_v)$ and $u_i^V X(t_w)$, of a given unit u_i^V are fed into the corresponding network pair in parallel. Here, $(t_v, t_w) = (t_a, t_b)$ for source domain, while $(t_v, t_w) = (t_c, t_d)$ for target domain. The latent features, $u_i^V z(t_v)$ and $u_i^V z(t_w)$, extracted from the input samples by the encoders are then passed into the corresponding regressor heads to predict the RULs, $u_i^V \hat{y}(t_v)$ and $u_i^V \hat{y}(t_w)$, according to the individual input samples. We aim to learn the latent feature representations that should (i) be discriminative for the RUL prediction task in the source domain, (ii) be transferable between source and target domains, and (iii) be aligned in such a way that the deviation of each predicted RUL pair matches the temporal distance (pseudo label, $t_v - t_w$) of its corresponding input sample pair. To this end, on the one hand, we train our model in a supervised manner using the ground-truth RUL labels in source domain (see Section 4.1). On the other hand, we force the learned feature representations to be indistinguishable between source and target domains by means of a domain alignment module (see Section 4.2). Furthermore, we train our model to additionally solve

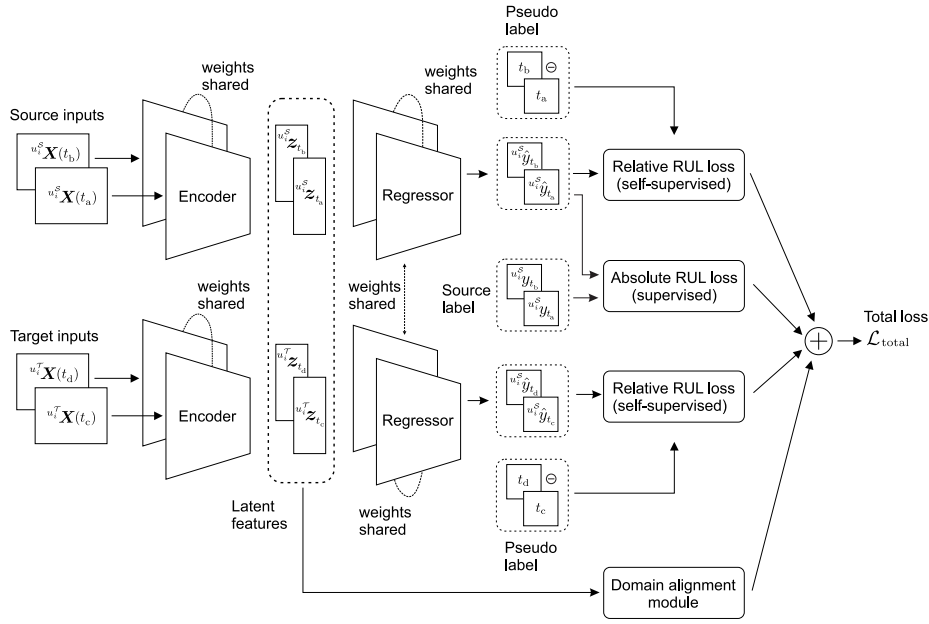


Fig. 2. Schematic illustration of the proposed SSDA framework (cf. Figs. 3–5).

a common pre-text task in a self-supervised manner in both the source and target domains to avoid the negative transfer effects caused by the semantic misalignment of the feature representations (see Section 4.3).

4.1. Supervised learning in source domain

Let $u_i X = (x_t^i)_{t=1}^{T_i} \in \mathbb{R}^{T_i \times M}$ be the condition data of a given unit u_i recorded during its lifetime by a monitoring system with M sensors, where T_i is the total lifetime of unit u_i counted from the time point when the monitoring system starts to track the condition of this unit. In the following, we will refer to $u_i X$ as the tracked condition data of unit u_i . At the current time step t_* , given the condition data recorded over the past L time steps (including the current one) of unit u_i

$$u_i X(t_*) = (x_{t_*+l}^i)_{l=-L+1}^0, \quad (7)$$

the task is to predict the RUL of unit u_i

$$u_i \hat{y}_{t_*} = G_r(G_f(u_i X(t_*))), \quad (8)$$

such that the predicted RUL is as close as possible to the actual RUL

$$u_i y_{t_*} = T_i - t_*. \quad (9)$$

In the source domain, where the actual RULs are available, the RUL prediction task described above can be interpreted as a typical supervised learning regression task. We can therefore train our model as usual by minimizing the source empirical loss w.r.t. the RMSE loss (see Fig. 3):

$$\mathcal{L}_{SL}(S; \theta_f, \theta_r) = \mathbb{E}_{\substack{u_i \in \mathcal{U}^S \\ t \in [L, T_i]}} [\mathcal{L}_{RMSE}(u_i \hat{y}_t, u_i y_t)], \quad (10)$$

where $\mathcal{U}^S$ is the set of units in the source domain. In practice, for a joint optimization of the overall loss including that of the pre-text (SSL) task as described in Section 4.3, in each forward pass, two input samples of the source domain are fed in parallel into the framework. Therefore, we formulate the corresponding RMSE loss used for the supervised learning part of our proposed framework as follows (see Fig. 2):

$$\mathcal{L}_{SL}(S; \theta_f, \theta_r) = \mathbb{E}_{\substack{u_i \in \mathcal{U}^S \\ (t_a, t_b) \in [L, T_i] \times [L, T_i]}} [\mathcal{L}_{RMSE}(u_i \hat{y}_{t_a}, u_i y_{t_a}) + \mathcal{L}_{RMSE}(u_i \hat{y}_{t_b}, u_i y_{t_b})]. \quad (11)$$

4.2. Feature-based domain alignment

A large number of UDA methods based on the alignment of feature representations have been developed over the last decade. They differ from each other mostly in the way the distribution discrepancy is measured. In this section, we describe two techniques of feature-based domain alignment, namely maximum mean discrepancy and adversarial domain alignment, which we deploy in this work for the domain alignment module of our SSDA framework (see Fig. 2).

Maximum Mean Discrepancy: First introduced in the context of two-sample test, maximum mean discrepancy (MMD) [58,59] was designed as a non-parametric measure to compare two distributions given their observed samples without the need for intermediately estimating the distribution densities. In domain adaptation, MMD is widely used as a metric to align the feature representations between the source and target domains [18,19,24,25,60]. Let $\mathcal{D}_{Z^S} = \{z_1^S, \dots, z_{N_S}^S\} \sim p_{Z^S}$ and $\mathcal{D}_{Z^T} = \{z_1^T, \dots, z_{N_T}^T\} \sim p_{Z^T}$ be the sets of latent features extracted from the inputs of source and target domains. As an integral probability metric, the MMD between distributions p_{Z^S} and p_{Z^T} is defined as the largest difference in expectations over functions of the unit ball of a reproducing kernel Hilbert space (RKHS) $\mathcal{H}_k$ endowed by a characteristic kernel k (cf. [25,58,59]):

$$\text{MMD}(p_{Z^S}, p_{Z^T}) := \sup_{\|f\|_{\mathcal{H}_k} \leq 1} (\mathbb{E}_{Z^S} [f(Z^S)] - \mathbb{E}_{Z^T} [f(Z^T)]). \quad (12)$$

More intuitively, the MMD between p_{Z^S} and p_{Z^T} can also be expressed as the RKHS (squared) distance between their kernel mean embeddings $\mu_{Z^S}(p_{Z^S}) := \mathbb{E}_{Z^S} [\phi(Z^S)]$ and $\mu_{Z^T}(p_{Z^T}) := \mathbb{E}_{Z^T} [\phi(Z^T)]$ (cf. [19,58,59]):

$$\begin{aligned} \text{MMD}(p_{Z^S}, p_{Z^T}) &= \left\| \mathbb{E}_{Z^S} [\phi(Z^S)] - \mathbb{E}_{Z^T} [\phi(Z^T)] \right\|_{\mathcal{H}_k}^2 \\ &= \mathbb{E}_{Z^S, Z^S} [k(Z^S, Z^S)] - 2 \mathbb{E}_{Z^S, Z^T} [k(Z^S, Z^T)] \\ &\quad + \mathbb{E}_{Z^T, Z^T} [k(Z^T, Z^T)]. \end{aligned} \quad (13)$$

Here, $\phi(Z) := k(Z, \cdot)$ denotes the implicit feature mapping with $k(Z, Z') = \langle \phi(Z), \phi(Z') \rangle_{\mathcal{H}_k}$, where $\langle \cdot, \cdot \rangle_{\mathcal{H}_k}$ represents the scalar product defined in $\mathcal{H}_k$. In fact, the choice of the characteristic kernel is crucial

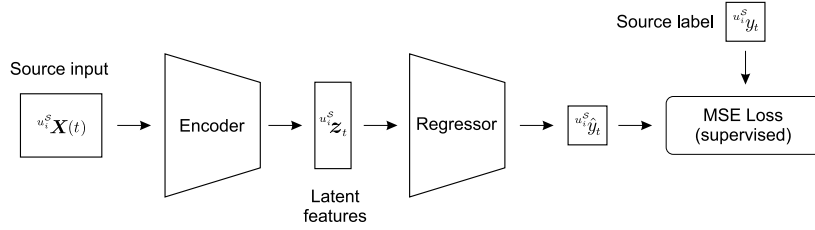


Fig. 3. Illustration of the supervised learning in the source domain. Here, only a single forward pass with one source input sample passed into the model is depicted. Note that, in practice, to facilitate an efficient implementation in compliance with the self-supervised learning part, two source input samples are fed into the framework in parallel during each forward pass (cf. Fig. 2).

for the robustness of the two-sample hypothesis test. As a universal kernel, the Gaussian kernel defined as $k(x, x') := \exp\left(-\frac{\|x-x'\|_2^2}{\gamma}\right)$, where γ is the kernel bandwidth, is widely used in practice to compute MMD. Minimizing the MMD while using a Gaussian kernel as the characteristic kernel can be interpreted as matching all moments of the two distributions of interest [61]. For the underlying task of RUL prediction, the MMD loss can be formulated more concretely as follows:

$$\begin{aligned} \mathcal{L}_{\text{MMD}}(S, \mathcal{T}) = & \mathbb{E}_{\substack{(u_i, u_{j'}) \in \mathcal{U}^S \times \mathcal{U}^S \\ (j, j') \in [L, T_i] \times [L, T_{j'}]}} [k(u_i z_j, u_{j'} z_{j'})] \\ & - \frac{2}{\mathbb{E}_{\substack{(u_i, u_{j'}) \in \mathcal{U}^S \times \mathcal{U}^T \\ (j, j') \in [L, T_i] \times [L, T_{j'}]}}} [k(u_i z_j, u_{j'} z_{j'})] \\ & + \mathbb{E}_{\substack{(u_i, u_{j'}) \in \mathcal{U}^T \times \mathcal{U}^T \\ (j, j') \in [L, T_i] \times [L, T_{j'}]}} [k(u_i z_j, u_{j'} z_{j'})]. \end{aligned} \quad (14)$$

Inspired by [19,61,62], instead of using a single Gaussian kernel, we use a mixture of d Gaussian kernels

$$k(Z, Z') := \sum_{\omega=1}^d k_{\omega}(Z, Z') = \exp\left(-\frac{\|Z - Z'\|_2^2}{\gamma_{\omega}}\right) \quad (15)$$

to compute MMD as a measure to align the distributions of source and target features.

Adversarial domain alignment: To realize the adversarial domain alignment, the base model composed of feature extractors G_f^S, G_f^T with parameters $\theta_f := (\theta_f^S, \theta_f^T)$ and RUL regressor G_r with parameters θ_r needs to be extended by a domain discriminator G_d parameterized by θ_d . As its name suggests, the domain discriminator $G_d : \mathcal{Z} \rightarrow \{0, 1\}$ serves as a classifier responsible for distinguishing whether a given feature vector z comes from the source (label 0) or the target domain (label 1). For the feature alignment, we aim to train the extended model towards a saddle point solution of the following minimax problem:

$$\begin{aligned} (\hat{\theta}_f, \hat{\theta}_r) = & \arg \min_{\theta_f, \theta_r} (\mathcal{L}_{\text{RUL}}(S, \mathcal{T}; \theta_f, \theta_r) - \lambda \mathcal{L}_d(S, \mathcal{T}; \theta_f, \hat{\theta}_d)), \\ \hat{\theta}_d = & \arg \max_{\theta_d} (\mathcal{L}_{\text{RUL}}(S, \mathcal{T}; \hat{\theta}_f, \hat{\theta}_r) - \lambda \mathcal{L}_d(S, \mathcal{T}; \hat{\theta}_f, \theta_d)), \end{aligned} \quad (16)$$

where $\mathcal{L}_{\text{RUL}}$ is the RUL-related loss and $\mathcal{L}_d$ is the (binary cross entropy) domain classification loss formulated as follows:

$$\begin{aligned} \mathcal{L}_d(S, \mathcal{T}; \theta_f, \theta_d) = & \mathbb{E}_{\substack{u_i \in \mathcal{U}^S \cup \mathcal{U}^T \\ i \in [L, T_i]}} \left[-u_i d_i \log(u_i \hat{d}_i) \right. \\ & \left. - (1 - u_i d_i) \log(1 - u_i \hat{d}_i) \right]. \end{aligned} \quad (17)$$

Here, $u_i \hat{d}_i = G_d(u_i z_i)$ denotes the predicted domain label of feature vector $u_i z_i$, and $u_i d_i$ is the corresponding ground-truth domain label:

$$u_i d_i = \begin{cases} 0 & \text{for } u_i \in \mathcal{U}^S, \\ 1 & \text{for } u_i \in \mathcal{U}^T. \end{cases} \quad (18)$$

Following [20,21,32], to realize the training towards the desired saddle point solution, we deploy a gradient reversal layer (GRL), denoted by R_{λ} , between the feature extractor G_f and the domain discriminator

G_d , i.e. $u_i \hat{d}_i = G_d(R_{\lambda}(u_i z_i))$. During the forward pass, the GRL acts as an identity function and has no effect on the original model. In contrast, if the backpropagation is passed through the GRL, the gradient w.r.t. the upstream layer parameters θ_f will be multiplied by $-\lambda$. Mathematically, the characteristic of the GRL can be formulated as follows:

$$R_{\lambda}(z) = z \quad (\text{forward pass}), \quad (19)$$

$$\frac{dR_{\lambda}}{dz} = -\lambda I \quad (\text{backpropagation}). \quad (20)$$

The corresponding adversarial domain alignment loss with GRL can then be expressed as

$$\begin{aligned} \mathcal{L}_{\text{adv}}(S, \mathcal{T}; \lambda, \theta_f, \theta_d) = & \mathbb{E}_{\substack{u_i \in \mathcal{U}^S \cup \mathcal{U}^T \\ i \in [L, T_i]}} \left[-u_i d_i \log(G_d(R_{\lambda}(u_i z_i))) \right. \\ & \left. - (1 - u_i d_i) \log(1 - G_d(R_{\lambda}(u_i z_i))) \right], \end{aligned} \quad (21)$$

where the ground-truth domain label $u_i d_i$ is defined as given in Eq. (18).

4.3. Self-supervised learning in both source and target domain

As mentioned above, aligning the marginal feature distributions of both domains and forcing the empirical source risk to be low are insufficient to guarantee a good generalization in the target domain. Our empirical observations have shown that the reason for this is the semantic misalignment of the feature representations between the source and target domains, where the target and source samples with vastly different ground-truth RULs are mismatched to each other in the feature space. To address this problem, we propose to further align the learned features at the semantic level by additionally solving a common self-supervised pre-text task in both the source and target domains. In the following, we will introduce the pre-text task as well as explain in more detail how we define the pseudo-labels needed to train this task.

Let $u_i X(t_a)$ and $u_i X(t_b)$ denote the input condition monitoring data corresponding to the time step t_a and t_b of unit u_i . Let $u_i y_{t_a}$ and $u_i y_{t_b}$ accordingly be the associated actual RULs. Our self-supervised pre-text task is now to predict the RUL $u_i \hat{y}_{t_a}$ and $u_i \hat{y}_{t_b}$ such that the relative predicted RUL

$$\begin{aligned} u_i \Delta \hat{y}(t_a, t_b) &:= u_i \hat{y}_{t_a} - u_i \hat{y}_{t_b} \\ &= (G_f \circ G_r)(u_i X(t_a)) - (G_f \circ G_r)(u_i X(t_b)) \end{aligned} \quad (22)$$

is as close as possible to the actual relative RUL

$$\begin{aligned} u_i \Delta y(t_a, t_b) &:= u_i y_{t_a} - u_i y_{t_b} \\ &= T_i - t_a - (T_i - t_b) \\ &= t_b - t_a. \end{aligned} \quad (23)$$

As can be seen in Eq. (23) above, for a given unit u_i , the actual relative RUL $u_i \Delta y(t_a, t_b)$ according to two arbitrary time steps t_a and t_b can be computed as their temporal distance without the need of knowing the actual RUL at the individual time steps. We can therefore use these relative RULs as labels to train the pre-text task, even for the target domain where no actual RULs are available (see Figs. 4 and 5).

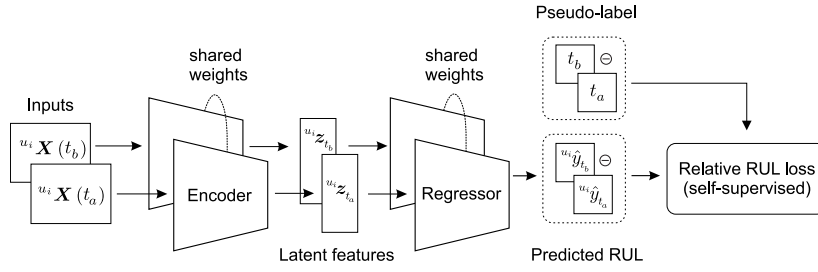


Fig. 4. Self-supervised learning in source and target domains (cf. Fig. 2).

Fig. 4 depicts the self-supervised learning in the source and target domains. In each forward pass, for each domain, two input samples, $u_i X(t_b)$ and $u_i X(t_a)$, of a given unit u_i are fed into the corresponding network pair in parallel. Each network pair contains two identical backbone networks. “Identical” here means not only that the networks have the same architecture but also that they actually share their weights. For each input sample, we receive a latent feature vector and a predicted RUL. To avoid the semantic misalignment of the feature representations between the source and target domains, we implicitly regularize the learned feature representations by additionally solving a self-supervised learning task that forces the deviation of the predicted RULs (relative RUL) to be consistent with the temporal distance of the input samples (pseudo-label). Fig. 5 illustrates the concept of the pre-text task. Given two condition monitoring data samples $u_i X(t_a)$ and $u_i X(t_b)$ of an arbitrary unit u_i according to two different time steps t_a and t_b , our task is now to predict the relative RUL of the unit w.r.t. these two input data samples. Labels for this pre-text task can be computed as the temporal distance of the input samples $t_b - t_a$ without the need for knowing the actual RUL label of each sample. Note that each input pair has to be sampled from the same unit.

In our proposed SSDA framework, the pre-text task of predicting the relative RULs should be solved in both the source and target domains. To this end, we train our model by minimizing the following self-supervised loss:

$$\mathcal{L}_{SSL}(S, \mathcal{T}; \theta_f, \theta_r) = \mathbb{E}_{\substack{u_i \in U^S \\ (t_a, t_b) \in [L, T] \times [L, T]}} [\mathcal{L}_{RMSE}(u_i \Delta \hat{y}(t_a, t_b), u_i \Delta y(t_a, t_b))] + \mathbb{E}_{\substack{u_i \in U^T \\ (t_c, t_d) \in [L, T] \times [L, T]}} [\mathcal{L}_{RMSE}(u_i \Delta \hat{y}(t_c, t_d), u_i \Delta y(t_c, t_d))], \quad (24)$$

where the predicted relative RULs $u_i \Delta \hat{y}$ and the actual relative RULs $u_i \Delta y$ are defined as given in Eq. (22) and (23), respectively. Note that different from the conventional concept of self-supervised learning, we do not pre-train our feature extractor separately for the pre-text task in advance. Instead, our self-supervised pre-text task is jointly trained along with other sub-tasks that aim at learning the domain-invariant features as well as minimizing the empirical source risk.

4.4. Overall loss function

The overall loss function can be formulated as follows:

$$\mathcal{L}_{total} = \mathcal{L}_{SL}(S) + \alpha \mathcal{L}_{SSL}(S, \mathcal{T}) + \beta \mathcal{L}_{UDA}(S, \mathcal{T}), \quad (25)$$

with α and β being the hyperparameters controlling the trade-off between the learning objectives. Here, the formulation of the loss for the supervised learning in source domain $\mathcal{L}_{SL}(S)$ and that of the loss for the self-supervised learning in both source and target domains $\mathcal{L}_{SSL}(S, \mathcal{T})$ are given in Eqs. (11) and (24), respectively. Depending on whether MMD or adversarial training is applied for the alignment of the marginal feature distributions, the unsupervised domain alignment loss $\mathcal{L}_{UDA}(S, \mathcal{T})$ here will be substituted by the MMD loss $\mathcal{L}_{MMD}(S, \mathcal{T})$ as given in Eq. (14) or the domain classification loss $\mathcal{L}_{adv}(S, \mathcal{T}; \lambda)$ as given in Eq. (21).

5. Experiments and results

5.1. Dataset

We use the benchmark C-MAPSS dataset [63,64] provided by NASA as a case study to evaluate the performance of our proposed method in comparison with other baseline methods. C-MAPSS has been widely used to evaluate methods for the task of RUL prediction in general as well as for domain adaptive RUL prediction in particular. The dataset is composed of four sub-datasets (FD001, FD002, FD003, FD004) containing the run-to-failure degradation data of aircraft turbofan engines. The data is provided in the form of multivariate time series representing the readings of 21 sensors, which are generated under multiple operating conditions and fault modes using a model-based simulation program called Commercial Modular Aero-Propulsion System Simulation. Statistics of the dataset are given in Table 1. Each sub-dataset is composed of one training and one test set. While the training sets contain run-to-failure condition monitoring data, the test sets include only partial degradation data that is terminated before the corresponding engines reach their end of life. The RUL labels are explicitly provided for the engines in the test sets of all sub-datasets. In contrast, the RUL labels are not explicitly provided for the engines in the training sets. As the training sets contain run-to-failure condition monitoring data, the RUL of a given engine at a given time step t_* can be computed as the difference between the last monitoring time step, i.e. the end of life, and the current time step t_* .

The difference in the operating conditions and fault modes results in the domain shift in the condition data of the sub-datasets. Fig. 6 depicts the UMAP representation [65] of the input data of the four sub-datasets, illustrating their distribution shift caused by the difference in the operating conditions and fault modes. UMAP is an efficient dimensionality reduction method that allows high-dimensional data to be visualized in a lower-dimensional space while preserving the original underlying data structure. In Fig. 6, each point is a two-dimensional UMAP representation of the corresponding 14-dimensional sensor readings at each monitoring time step. For each sub-dataset, the marginal distributions of the data points along each embedding component are also plotted at the top and right edges. As can be seen here, the embeddings of the data samples from each sub-dataset with the same operating conditions and fault modes tend to be close to each other and form point clusters. In contrast, data samples with different operating conditions and fault modes tend to form different clusters in different regions. For example, the data points of FD001 with one operating condition form one cluster in the embedding space, while six clusters can be observed for the data points of FD002 corresponding to six operating conditions. FD003 also contains data of the same operating condition as that of FD001, but due to the difference in the fault modes, the data point clusters of FD001 and FD003 are distributed in different regions. The difference in the distribution of the sensor readings across the sub-datasets observed in the embedding space confirms the existence of the domain shift problem in the data used in this work.

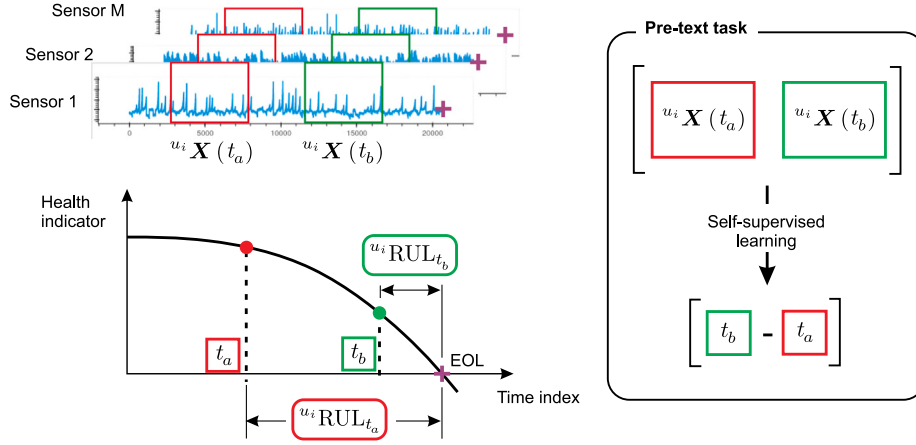


Fig. 5. Pre-text task for self-supervised learning in source and target domains (cf. Fig. 4).

Table 1
Statistics of the C-MAPSS dataset.

Subset	FD001	FD002	FD003	FD004
# training engine units	100	260	100	249
# test engine units	100	59	100	248
# operating conditions	1	6	1	6
# fault modes	1	1	2	2
# training samples	17 731	46 219	21 820	54 028
# test samples	100	259	100	248

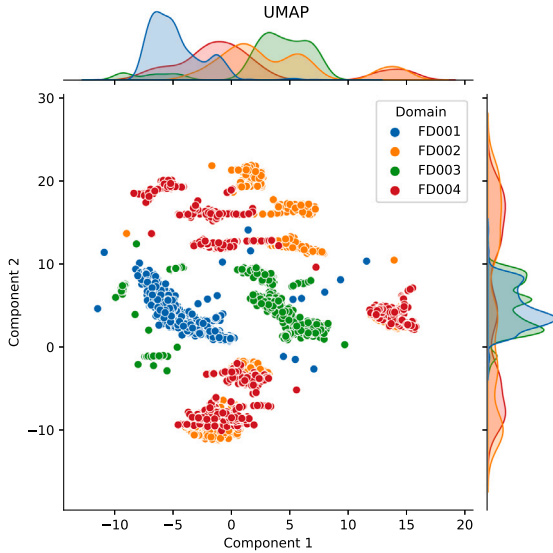


Fig. 6. UMAP representation of the input features illustrating the distribution shift (covariate shift) across domains. Each domain here corresponds to a sub-dataset of the C-MAPSS dataset. These sub-datasets differ from each other in term of the number of operating conditions as well as fault modes (see Table 1).

5.2. Evaluation metrics

For ease of comparison with other existing studies, we also use RMSE and the scoring function proposed by NASA [64], which are widely used as evaluation metrics for the task of RUL prediction on the C-MAPSS dataset, to measure the performance of our methods. Let $|U|$ be the number of units under test and $\rho_i = \hat{y}_i - y_i$ be the prediction error according to the test unit u_i . The RSME and the scoring function

can be then formulated as follows:

$$\text{RMSE} = \frac{1}{|U|} \sqrt{\sum_{i=1}^{|U|} \|\rho_i\|_2^2}, \quad (26)$$

$$\text{Score} = \sum_{i=1}^{|U|} s_i, \quad \text{with } s_i = \begin{cases} \exp\left(-\frac{\rho_i}{13}\right) - 1, & \text{if } \rho_i < 0 \\ \exp\left(\frac{\rho_i}{10}\right) - 1, & \text{if } \rho_i \geq 0. \end{cases} \quad (27)$$

Unlike the RMSE, the scoring function is an asymmetric metric that penalizes overestimation more than underestimation.

5.3. Implementation details

5.3.1. Data preparation

In the C-MAPSS dataset, the condition monitoring data has the form of multivariate time series, corresponding to the readings of 21 sensors in total. However, for some of the sensors, their readings are almost constant during the whole degradation trajectory, which are not informative for the RUL prediction. Therefore, in this work, we only take the measurements of 14 out of 21 sensors as the input data, which are not almost constant in any of the sub-datasets (cf. [33,35]). The measurements of the following sensors are thus dropped from the available dataset: S1, S5, S6, S10, S16, S18, S19. Furthermore, we set the sliding window size L to 30 for all subsets to keep the number of operating cycles which are seen by the predictive models to make predictions, consistent across all the subsets (cf. [32,33,35]).

One of the main factors affecting the domain shift between the sub-datasets is the difference in the operating condition. We observed that the range of the sensor measurement values varies not only across the different sensors but even for the same sensor, its measurement value range also varies across the different operational condition settings. When working with DL models, the input features should be typically normalized. This has proven to have positive effects on the convergence behavior of the training process. Regarding the domain adaptive RUL prediction on the C-MAPSS dataset, two main kinds of preprocessing strategies for the input data can be found in the literature: (1) normalizing the input features individually, i.e. within each sensor [32,66]; and (2) normalizing the input features individually but with respect to the operating condition, i.e., the sensor readings are grouped first according to the operating condition before the normalization is applied [33,35]. The second type of normalization strategy described above will have a significant effect on the extent of the domain gap between the subsets. This will accordingly affect the performance of the domain adaptation methods. Thus, as an effort to keep the performance comparisons as fair as possible, we report the evaluation metrics of our models according

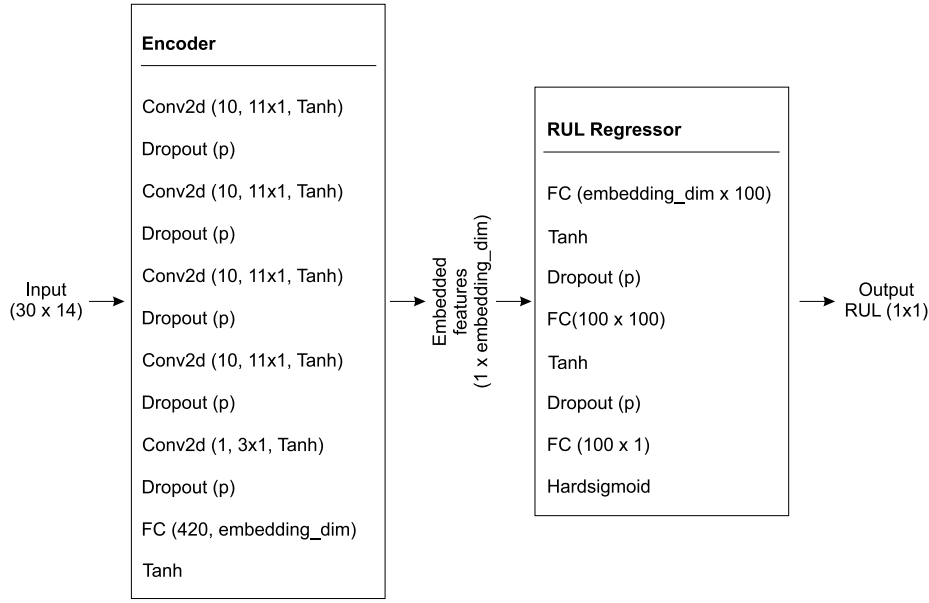


Fig. 7. Architecture of the backbone network.

to both of these normalization strategies and compare them with those of other methods that share the same concept of the normalization strategies. In fact, there are several further minor differences in the implementation details of the normalization strategies among the methods under consideration, for example, the scaler type: min-max scaler or the z-score scaler; and the normalization range: $[0, 1]$ or $[-1, 1]$. Our empirical observations, however, indicated that these differences have a merely insignificant effect on the model performance. In practice, for each sensor of each subset, we deploy a min-max scaler to normalize the input features to the range $[-1, 1]$ for both the normalization strategies. Moreover, following other works in the literature [7,33,35], we set a maximum RUL threshold for the cycles when the unit considered is still in a healthy state ($RUL_{max} = 125$). The actual RUL labels in the dataset were also normalized to the range $[0, 1]$ based on the pre-defined maximum RUL.

5.3.2. Network architecture

In principle, our SSDA framework does not require a specific structure for the backbone predictive model at the architecture level (see Fig. 2). For example, it does not matter if the encoder or feature extractor is implemented by a CNN-based or LSTM-based neural network. What is more important is that we need to have two sub-networks designed in such a way that at the functional level, one of them serves as an encoder to extract relevant features from the input data, while the other one is responsible to predict the RUL based on the features extracted. Fig. 7 depicts the architecture of the backbone predictive network implemented in this work for our proposed SSDA framework. The basis of the architecture is inspired by the DCNN proposed in [7], which also addressed the RUL prediction task on the C-MAPSS dataset but not for the domain adaptation learning scenario. We chose this architecture because it is a simple but efficient one that achieved competitive state-of-the-art performance on the C-MAPSS dataset while not suffering from high computational cost as in the case of LSTM-based models [5,7]. In our implementation, the encoder is composed of five convolutional layers with Tanh activation stacked sequentially, each consists of 10 kernels of size 11×1 , except the for that last one that contains one kernel of size 3×1 . The features extracted by the convolutional layers are then transformed by a FC layer to a feature vector of a pre-defined embedding dimension. The embedded feature vector is then fed into a regression head composed of three FC layers to compute the predicted RUL. The Hardsigmoid output activation is used

to ensure that the predicted RUL is normalized to the range $[0, 1]$. Note that the convolutional layers used here are two-dimensional but each kernel is one-dimensional, i.e. the 1D kernels are shifted along both dimensions of the input map. This allows the model to learn features along the time dimension separately from each time series, rather than learning features that are blurred across multiple time series by using 2D kernels.

5.3.3. Training details

We conducted experiments on 12 domain adaptation scenarios, each with two variants of preprocessing pipelines that differ from each other in the way the input data is normalized (see Section 5.3.1). In each domain adaptation scenario, one of the four subsets in C-MAPSS was used as the source domain dataset while one of the remaining three subsets served as the target domain dataset. For example, if FD001 was used as the source domain dataset and FD002 was the target domain dataset, then the adaptation scenario would be $FD001 \rightarrow FD002$. For training and validation, only the inputs and labels from the training data of the source domain dataset as well as the inputs from the training data of the target domain dataset were used. As a reminder, all training sets of C-MAPSS contain run-to-failure condition monitoring data. Thus, if all available degradation (i.e. run-to-failure) data in the training set of the target domain is used for model development, as in the experiments reported in Sections 5.4 and 5.5, the corresponding evaluation results should be interpreted as the upper bound of the performance. The test data of the target domain dataset in each domain adaptation scenario was used only to evaluate the performance of the developed models. For a given subset used as the source domain, its test engines were not used, neither for model development nor for performance evaluation.

During training, the available degradation sequence of each engine was not fed into the neural network all at once in the ascending temporal order. In contrast, the available degradation sequence of each engine was sliced into overlapping sliding windows with a window size of 30 and a hop size of 1. In each forward pass during training, only two windows from the degradation data of a given engine were randomly sampled and fed as input to the proposed framework (cf. Sections 4.3 and 5.3.1). Mini-batch gradient descent with a batch size of 128 was applied during training. We trained our models for 100 epochs using an Adam optimizer with an initial learning rate η and a weight decay ω towards minimizing the overall loss function defined in Eq. (25). Here, the multiplicative coefficient of the self-supervised loss term α



Fig. 8. Visualization of the experimental results in comparison with other state-of-the-art methods. Here, Score is not plotted for NS = 1, since among the methods under consideration, this metric is only available in our proposed SSDA (cf. Table 2).

was set to 1. If the MMD-based feature alignment was deployed for the UDA part, the multiplicative coefficient of the UDA loss term β was tuned as a hyperparameter. In contrast, in the experiments where the adversarial feature alignment with GRL was deployed, we fixed β to 1 while tuning the GRL coefficient λ as a hyperparameter. Furthermore, for the MMD loss, we used a mixture of d Gaussian kernels as defined in Eq. (15), where $d = 5$ and γ_ω was varied between $2^{-2}\gamma$ and $2^2\gamma$ with a multiplicative step size of 2 and the base bandwidth γ was set to the mean pairwise distance of the data samples in each training batch (cf. [19]). We performed multi-objective hyperparameter optimizations using a Bayesian optimization framework called Optuna with a TPE sampler [67]. The hyperparameter search space was: $\eta \sim \log U[1e - 5, 0.1)$, $\omega \sim U[0, 1e - 2)$, β or $\lambda \sim U[0, 10)$. These hyperparameters were optimized on the basis of two objectives. The first one was defined as the sum of the supervised and self-supervised loss terms, while the second one was defined as the MMD loss or the domain confusion loss [68], depending on whether MMD or adversarial training with GRL was used for the unsupervised domain alignment part (see Section 4.4). Furthermore, we employed K -fold cross-validation with $K = 5$. The number of trials for HPO was set to 100.

5.4. Comparison with other domain adaptation methods

Table 2 and Fig. 8 show the experimental results of our proposed methods in comparison with the performance of other state-of-the-art methods that also deal with the task of domain adaptive RUL prediction on the C-MAPSS dataset. Here, the methods under evaluation are grouped according to the normalization strategy (NS) applied (see Section 5.3.1). As can be seen here, our SSDA significantly outperforms the other methods under consideration in most of the domain adaptation scenarios in terms of both RMSE and Score. Among 24 evaluation scenarios, our proposed SSDA shows the best performance in 18 scenarios in terms of RMSE and in 19 scenarios in terms of Score. For NS = 1, we obtain a mean RMSE of 24.1 and a mean score of 4798, while the mean RMSE of the current best method (DRR) is 36.8. For NS = 2, our proposed method achieves a mean RMSE of 18.1 and a mean score of 1219, while these values of the current best method (MTLTran) are 19.2 and 3518. On average, compared to the current state of the art for each normalization strategy, we achieve an improvement of 20.1% and 51.2% for RMSE and Score, respectively. In particular, in the most challenging adaptation scenario, namely, FD001 $\rightarrow$ FD004 (NS = 1), where the source and target subsets differ from each other in both the number of operating conditions (1 $\rightarrow$ 6) and the

Table 2
Experimental results in comparison with other state-of-the-art methods.

NS	Method	Source	FD001			FD002		
		Target	FD002	FD003	FD004	FD001	FD003	FD004
(1)	CORAL [22,32]	RMSE ↓	77.50 ± 4.60	69.60 ± 5.20	84.60 ± 7.00	80.90 ± 9.40	79.80 ± 10.10	43.60 ± 3.60
		Score ↓	n.a.	n.a.	n.a.	n.a.	n.a.	n.a.
	DANN [32]	RMSE ↓	48.62 ± 6.83	45.87 ± 3.58	43.82 ± 4.15	28.10 ± 5.03	37.46 ± 1.54	31.85 ± 1.65
		Score ↓	n.a.	n.a.	n.a.	n.a.	n.a.	n.a.
	DRR [36]	RMSE ↓	43.40 ± 3.20	24.50 ± 2.10	45.10 ± 3.80	34.60 ± 5.30	43.20 ± 5.80	33.10 ± 2.50
		Score ↓	n.a.	n.a.	n.a.	n.a.	n.a.	n.a.
	SSDA (ours)	RMSE ↓	26.02 ± 0.62	18.54 ± 0.72	31.25 ± 0.57	21.05 ± 0.84	21.04 ± 1.74	32.11 ± 0.59
		Score ↓	7713 ± 3205	1424 ± 147	12466 ± 7534	817 ± 168	1135 ± 96	9501 ± 642
	RULDDA [32,35]	RMSE ↓	24.08 ± n.a.	43.08 ± n.a.	45.7 ± n.a.	23.91 ± n.a.	47.26 ± n.a.	45.17 ± n.a.
		Score ↓	2684 ± n.a.	10259 ± n.a.	26981 ± n.a.	2430 ± n.a.	12756 ± n.a.	25738 ± n.a.
(2)	ADARUL [33]	RMSE ↓	19.87 ± 0.27	39.74 ± 0.32	31.78 ± 0.36	14.33 ± 0.14	32.60 ± 0.39	34.35 ± 0.67
		Score ↓	4426 ± n.a.	10711 ± n.a.	12369 ± n.a.	455 ± n.a.	3449 ± n.a.	12227 ± n.a.
	CADA [35]	RMSE ↓	19.52 ± n.a.	39.58 ± n.a.	31.23 ± n.a.	13.82 ± n.a.	33.65 ± n.a.	33.71 ± n.a.
		Score ↓	2122 ± n.a.	8415 ± n.a.	11577 ± n.a.	351 ± n.a.	5213 ± n.a.	15106 ± n.a.
	DIDRLSTM [34]	RMSE ↓	29.40 ± 0.30	20.90 ± 0.10	34.40 ± 0.40	16.10 ± 0.10	21.20 ± 0.50	34.00 ± 0.10
		Score ↓	4822 ± 440	470 ± 22	5538 ± 469	213 ± 16	474 ± 34	5471 ± 223
	MTLTrans [44]	RMSE ↓	16.63 ± n.a.	20.83 ± n.a.	21.76 ± n.a.	13.67 ± n.a.	20.60 ± n.a.	22.18 ± n.a.
		Score ↓	1618 ± n.a.	2286 ± n.a.	4714 ± n.a.	331 ± n.a.	1510 ± n.a.	3860 ± n.a.
	SSDA (ours)	RMSE ↓	18.13 ± 0.91	18.37 ± 0.39	22.14 ± 1.20	14.60 ± 0.29	17.27 ± 0.39	21.16 ± 0.28
		Score ↓	1311 ± 55	1373 ± 248	2052 ± 158	345 ± 33	1156 ± 203	2125 ± 113
		Source	FD003			FD004		
		Target	FD001	FD002	FD004	FD001	FD002	FD003
(1)	CORAL [22,32]	RMSE ↓	26.50 ± 1.90	75.60 ± 9.50	77.20 ± 9.10	94.00 ± 8.80	30.90 ± 1.40	68.60 ± 11.20
		Score ↓	n.a.	n.a.	n.a.	n.a.	n.a.	n.a.
	DANN [32]	RMSE ↓	31.74 ± 9.37	44.62 ± 1.21	47.94 ± 5.78	31.54 ± 2.42	24.93 ± 1.82	27.84 ± 2.69
		Score ↓	n.a.	n.a.	n.a.	n.a.	n.a.	n.a.
	DRR [36]	RMSE ↓	18.30 ± 1.20	48.80 ± 3.00	52.50 ± 2.70	35.30 ± 3.10	24.60 ± 1.60	38.70 ± 3.20
		Score ↓	n.a.	n.a.	n.a.	n.a.	n.a.	n.a.
	SSDA (ours)	RMSE ↓	16.80 ± 0.99	25.05 ± 1.63	32.42 ± 2.51	19.94 ± 1.10	24.55 ± 1.60	20.46 ± 0.38
		Score ↓	582 ± 131	4430 ± 888	11897 ± 2938	802 ± 127	5568 ± 1027	1243 ± 314
	RULDDA [32,35]	RMSE ↓	27.15 ± n.a.	30.42 ± n.a.	31.82 ± n.a.	32.37 ± n.a.	27.54 ± n.a.	23.31 ± n.a.
		Score ↓	2931 ± n.a.	6754 ± n.a.	5775 ± n.a.	13377 ± n.a.	4937 ± n.a.	1679 ± n.a.
(2)	ADARUL [33]	RMSE ↓	19.97 ± 0.45	23.47 ± 0.61	26.33 ± 0.88	37.89 ± 0.17	28.77 ± 0.34	14.13 ± 0.16
		Score ↓	1391 ± n.a.	20764 ± n.a.	48748 ± n.a.	121677 ± n.a.	62215 ± n.a.	537 ± n.a.
	CADA [35]	RMSE ↓	19.54 ± n.a.	19.33 ± n.a.	20.61 ± n.a.	20.10 ± n.a.	18.50 ± n.a.	14.49 ± n.a.
		Score ↓	1451 ± n.a.	5257 ± n.a.	3219 ± n.a.	1840 ± n.a.	4460 ± n.a.	682 ± n.a.
	DIDRLSTM [34]	RMSE ↓	18.88 ± 0.40	29.40 ± 0.40	32.30 ± 0.50	21.30 ± 0.30	28.90 ± 0.20	16.90 ± 0.10
		Score ↓	442 ± 45	4127 ± 214	4123 ± 448	553 ± 29	3483 ± 133	249 ± 12
	MTLTrans [44]	RMSE ↓	21.22 ± n.a.	23.79 ± n.a.	21.04 ± n.a.	18.44 ± n.a.	16.86 ± n.a.	13.56 ± n.a.
		Score ↓	1890 ± n.a.	9098 ± n.a.	13261 ± n.a.	1096 ± n.a.	2189 ± n.a.	383 ± n.a.
	SSDA (ours)	RMSE ↓	16.58 ± 0.60	19.36 ± 0.48	20.07 ± 0.54	16.35 ± 0.71	17.76 ± 0.22	15.85 ± 0.45
		Score ↓	472 ± 16	1477 ± 71	2014 ± 220	447 ± 55	1376 ± 30	482 ± 69

number of fault modes ($1 \rightarrow 2$), our SSDA is still able to achieve an improvement of 28.7% in terms of RMSE.

5.5. Ablation studies

To quantify the contribution of individual components of our proposed SSDA framework, we performed ablation studies and report their results in Table 3 as well as in Fig. 10. In the ablation studies, we compared the performance of the following methods while using the same network architecture as described in Section 5.3.2.

Source-only (w/o DA): The predictive models were trained in a standard supervised manner using only source labeled data, where no domain adaptation technique was applied.

UDA-based (w/o SS): The predictive models were trained using MMD or adversarial feature alignment with GRL as the UDA technique, but without employing the proposed self-supervised learning algorithm.

SSDA-based (ours): In SSDA-MMD and SSDA-GRL, all the network components were trained jointly from scratch, i.e. with random initialization, using the overall loss function as described in Section 4.4, where MMD and adversarial training with GRL were employed for the domain alignment module, respectively. Furthermore, we used the same encoder for both the source and target domains ($G_f^S = G_f^T$). In contrast, in SSDA-MMD-I, the source encoder G_f^S and the shared

regressor G_r were first trained in a standard supervised manner using only source labeled data. The pre-trained source encoder was then used as an initialization for the target encoder G_f^T . In the final training stage, we trained the target encoder using the total loss function while fixing the pre-trained weights of the source encoder and the regressor module ($G_f^T \neq G_f^S$).

It is obvious that “Source-only” suffers from poor prediction accuracy in most of the evaluation scenarios, except for the ones with a minimal domain gap between the source and target data, e.g. FD002 $\rightarrow$ FD001 (NS = 2). Furthermore, as can be seen here, classical UDA-based methods do not always perform better than “Source-only”, confirming that aligning only the marginal feature distributions can lead to negative transfer, particularly when the domain gap is significant, e.g. for the evaluation scenario FD002 $\rightarrow$ FD001 (NS = 1). In principle, there is no consistent evidence indicating whether using MMD or adversarial training with GRL is better in terms of prediction accuracy. However, we observed that training models using MMD loss tended to show more stable convergence behavior. Notably, our proposed SSDA-based methods show superior performance over both the “Source-only” and UDA-based baselines in all evaluation scenarios. This empirical observation indicates the significant advantages of additionally incorporating the intrinsic information of the target data into the domain

Table 3
Experimental results of the ablation studies.

NS	Method	Source Target	FD001			FD002		
			FD002	FD003	FD004	FD001	FD003	FD004
(1)	Source only (w/o DA)	RMSE ↓	52.05 ± 1.78	44.68 ± 8.14	52.80 ± 0.66	33.68 ± 0.46	65.37 ± 0.52	40.46 ± 1.24
		Score ↓	99 856 ± 26 777	12 218 ± 10 163	78 593 ± 4102	11 312 ± 1567	549 301 ± 60 527	22 448 ± 3604
	UDA-MMD (w/o SS)	RMSE ↓	54.64 ± 0.20	42.40 ± 6.96	53.62 ± 0.37	40.35 ± 0.03	39.93 ± 0.04	41.40 ± 1.35
		Score ↓	283 220 ± 35 283	141 064 ± 113 350	288 599 ± 42 925	4712 ± 4	5232 ± 9	23 896 ± 3493
	UDA-GRL (w/o SS)	RMSE ↓	71.61 ± 0.76	38.08 ± 1.64	48.08 ± 0.66	38.44 ± 0.46	48.26 ± 1.05	34.20 ± 0.40
		Score ↓	388 466 ± 23 453	43 326 ± 13 060	188 730 ± 34 414	4319 ± 175	26 946 ± 4258	13 414 ± 1872
	SSDA-GRL (ours)	RMSE ↓	26.02 ± 0.62	21.78 ± 2.10	33.56 ± 1.54	23.83 ± 1.06	24.58 ± 4.36	32.39 ± 1.24
		Score ↓	7713 ± 3205	1597 ± 136	11 224 ± 4026	1076 ± 208	2845 ± 309	10 702 ± 1696
	SSDA-MMD (ours)	RMSE ↓	27.81 ± 5.42	19.57 ± 1.16	31.25 ± 0.57	25.83 ± 0.70	31.39 ± 0.86	34.58 ± 2.84
		Score ↓	10 623 ± 1371	1475 ± 248	12 466 ± 7534	1868 ± 88	6179 ± 1601	15 246 ± 5455
	SSDA-MMD-I (ours)	RMSE ↓	26.17 ± 1.38	18.54 ± 0.72	33.13 ± 1.16	21.05 ± 0.84	21.04 ± 1.74	32.11 ± 0.59
		Score ↓	4080 ± 417	1424 ± 147	13 608 ± 3505	817 ± 168	1135 ± 96	9501 ± 642
(1)	Source only (w/o DA)	RMSE ↓	31.52 ± 3.33	65.28 ± 2.22	61.06 ± 0.90	38.15 ± 1.63	30.99 ± 0.94	54.63 ± 0.42
		Score ↓	7766 ± 2905	1 720 992 ± 182 440	1 144 778 ± 128 513	19 664 ± 2697	17 145 ± 4728	984 612 ± 35 312
	UDA-MMD (w/o SS)	RMSE ↓	32.51 ± 7.47	64.49 ± 2.17	63.99 ± 1.26	40.03 ± 0.03	28.56 ± 0.49	39.34 ± 0.02
		Score ↓	4165 ± 2008	1 552 157 ± 272 769	1 126 705 ± 99 544	5078 ± 29	9636 ± 2833	5521 ± 12
	UDA-GRL (w/o SS)	RMSE ↓	34.46 ± 3.08	61.45 ± 1.03	57.06 ± 2.48	36.62 ± 0.54	26.82 ± 0.55	55.44 ± 0.16
		Score ↓	9626 ± 3181	781 517 ± 233 519	708 867 ± 168 744	4545 ± 242	4839 ± 564	1 023 033 ± 818
	SSDA-GRL (ours)	RMSE ↓	17.93 ± 1.86	35.32 ± 9.11	34.91 ± 1.85	21.21 ± 0.48	24.55 ± 1.60	21.64 ± 0.74
		Score ↓	617 ± 81	15 503 ± 10 441	14 817 ± 5612	1138 ± 118	5568 ± 1027	2168 ± 384
	SSDA-MMD (ours)	RMSE ↓	17.43 ± 0.39	33.31 ± 1.60	36.50 ± 1.7	21.59 ± 2.46	24.92 ± 0.86	26.57 ± 1.63
		Score ↓	614 ± 102	12 366 ± 2702	23 389 ± 5882	985 ± 281	8569 ± 2985	3993 ± 1638
	SSDA-MMD-I (ours)	RMSE ↓	16.80 ± 0.99	25.05 ± 1.63	32.42 ± 2.51	19.94 ± 1.10	25.19 ± 0.37	20.46 ± 0.38
		Score ↓	582 ± 131	4430 ± 888	11 897 ± 2938	802 ± 127	3256 ± 204	1243 ± 314
(2)	Source only (w/o DA)	RMSE ↓	22.46 ± 1.69	36.54 ± 2.08	30.55 ± 1.29	14.83 ± 0.45	38.26 ± 1.01	45.92 ± 2.81
		Score ↓	2441 ± 635	4513 ± 1371	9906 ± 708	375 ± 99	5274 ± 1037	30 113 ± 9423
	UDA-MMD (w/o SS)	RMSE ↓	22.16 ± 1.67	40.87 ± 2.33	35.62 ± 3.97	15.10 ± 0.70	22.49 ± 1.71	34.04 ± 3.26
		Score ↓	2375 ± 667	18 891 ± 6957	14 253 ± 6190	416 ± 76	1248 ± 218	8313 ± 2834
	UDA-GRL (w/o SS)	RMSE ↓	24.11 ± 1.91	37.75 ± 6.19	44.56 ± 0.02	19.04 ± 0.90	50.15 ± 9.22	44.16 ± 4.14
		Score ↓	2565 ± 685	20 797 ± 17 629	16 113 ± 26	583 ± 107	26 804 ± 23 072	18 130 ± 8484
	SSDA-GRL (ours)	RMSE ↓	18.35 ± 0.49	20.81 ± 1.04	22.65 ± 0.92	17.85 ± 0.46	19.39 ± 0.61	23.29 ± 0.86
		Score ↓	1396 ± 113	1438 ± 317	2246 ± 191	696 ± 80	1711 ± 111	2571 ± 265
	SSDA-MMD (ours)	RMSE ↓	18.13 ± 0.91	19.57 ± 1.16	23.18 ± 0.69	17.58 ± 0.24	19.33 ± 0.59	23.09 ± 0.87
		Score ↓	1310 ± 55	1475 ± 248	2720 ± 395	569 ± 64	1315 ± 212	2730 ± 289
	SSDA-MMD-I (ours)	RMSE ↓	18.91 ± 0.57	18.37 ± 0.39	22.14 ± 1.20	14.60 ± 0.29	17.27 ± 0.39	21.16 ± 0.28
		Score ↓	1366 ± 45	1373 ± 248	2052 ± 158	345 ± 33	1156 ± 203	2125 ± 113
(2)	Source only (w/o DA)	RMSE ↓	31.02 ± 3.17	34.06 ± 1.68	28.75 ± 3.10	19.72 ± 2.72	18.12 ± 0.63	19.02 ± 1.72
		Score ↓	7482 ± 3201	165 064 ± 112 822	114 769 ± 112 032	1083 ± 454	1915 ± 354	1234 ± 491
	UDA-MMD (w/o SS)	RMSE ↓	16.89 ± 0.37	21.97 ± 0.29	21.91 ± 0.88	17.66 ± 0.34	19.37 ± 1.15	17.68 ± 0.79
		Score ↓	3823 ± 2699	3198 ± 519	2155 ± 197	572 ± 36	1830 ± 301	794 ± 260
	UDA-GRL (w/o SS)	RMSE ↓	44.06 ± 5.37	41.86 ± 8.00	36.54 ± 4.90	34.37 ± 5.19	32.69 ± 5.20	24.10 ± 3.43
		Score ↓	12 375 ± 537	37 697 ± 23 276	21 093 ± 13 272	5426 ± 3433	10 600 ± 4554	1246 ± 485
	SSDA-GRL (ours)	RMSE ↓	18.49 ± 1.85	19.90 ± 1.95	21.22 ± 0.64	19.41 ± 1.43	20.10 ± 1.35	17.93 ± 1.31
		Score ↓	830 ± 407	1512 ± 146	2592 ± 507	697 ± 113	1669 ± 194	1060 ± 379
	SSDA-MMD (ours)	RMSE ↓	17.43 ± 0.39	19.54 ± 1.74	20.82 ± 0.64	17.63 ± 0.75	19.05 ± 0.68	17.65 ± 1.24
		Score ↓	613 ± 102	1774 ± 503	2029 ± 107	597 ± 118	1654 ± 245	976 ± 581
	SSDA-MMD-I (ours)	RMSE ↓	16.58 ± 0.60	19.36 ± 0.48	20.07 ± 0.54	16.35 ± 0.71	17.76 ± 0.22	15.85 ± 0.45
		Score ↓	472 ± 16	1477 ± 71	2014 ± 220	447 ± 55	1376 ± 30	482 ± 69

adaptation process in a self-supervised manner according to our proposed method. Among the SSDA-based methods, SSDA-MMD-I achieves the best prediction accuracy in most of the evaluation scenarios. This suggests that it is beneficial to use the source encoder pre-training on the source labeled data as an initialization for the target encoder before further training it by applying the proposed SSDA.

See Fig. 9 for an exemplary illustration of the features learned by different methods, each is representative for one of the three method groups considered in the ablation studies. The characteristics of the learned features, as seen here, once again confirm the limitations of the Source-only and the classical UDA-based methods, while demonstrating the superiority of our proposed SSDA. (a) Without domain adaption, a

significant distribution shift can be observed in the latent feature space of the source and target domains. A clear trend of the degradation trace can only be seen in the feature representations of the source domain. (b) Without the proposed self-supervision, the learned features of the source and target domains are semantically misaligned, while the marginal feature distributions are well aligned. Here, for example, the target samples with low RUL labels (blue points at the bottom left of the right sub-figure) are mismatched to the source samples with high RUL labels (yellow point at the bottom left of the left sub-figure). (c) The source and target features are well-aligned w.r.t. both marginal, $p^S(z^S) \approx p^T(z^T)$, and conditional distribution, $p^S(y|z^S) \approx p^T(y|z^T)$.

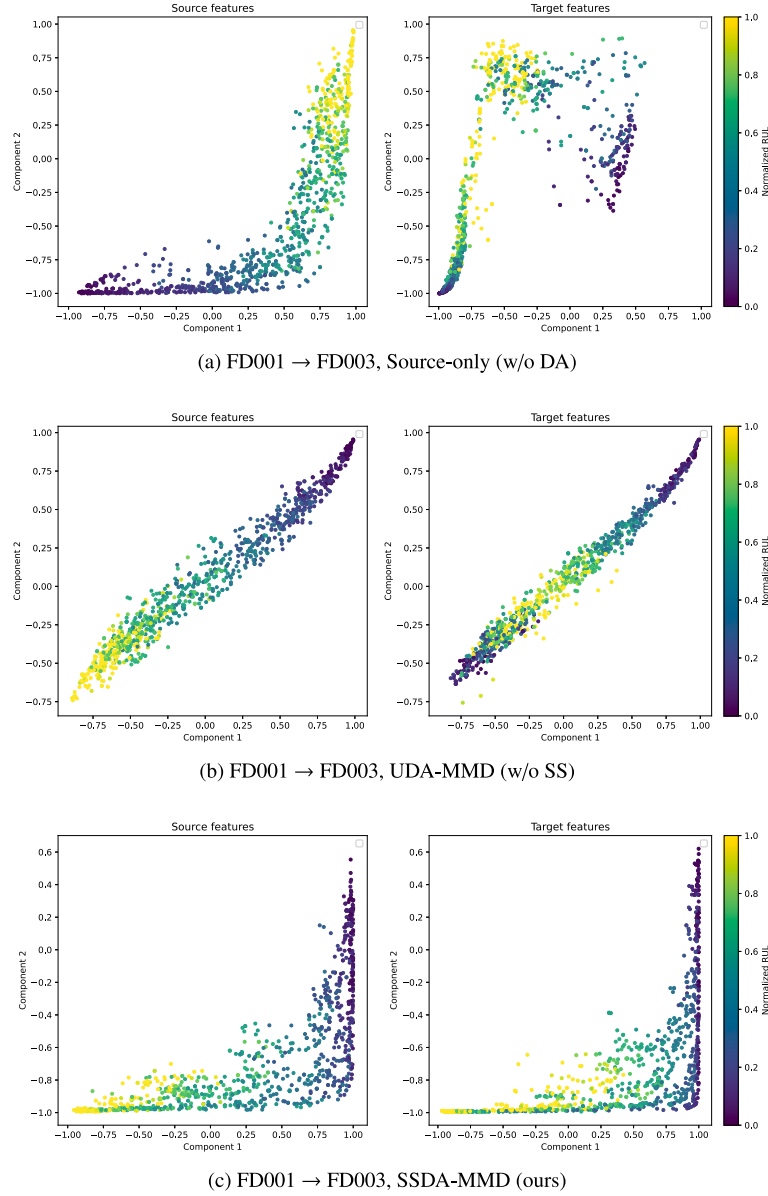


Fig. 9. Exemplary illustration of the features learned by our proposed method compared to other baseline methods.

A clear trend of the degradation trace can be seen in the feature representations of both domains.

5.6. Impact of target data availability

To be consistent with the experimental setup of most existing studies on RUL prediction with domain adaptation [32–35], for the experimental results reported in Sections 5.4 and 5.5, our models were built using the original training sets including run-to-failure degradation data of the source and target domains (see Section 5.1 and Table 1). In practice, however, the run-to-failure data in the target domain is usually scarce or even unavailable. Therefore, it is worth investigating how the proposed framework performs when only partial degradation data of the target domain is available. For this purpose, we assume that the sensor readings of each training unit in the target domain are only recorded up to some timestep before the unit reaches its end of life. This means that for each training unit of the target domain, a given proportion of the last time steps is truncated from the full run-to-failure degradation data. We considered six different scenarios of target data availability with the truncation level set to 0%, 5%, 25%, 50%,

75%, and 100%. Note that a truncation level of 100% means that no target-domain data is available for training (Source-only), so no domain adaptation methods can be applied.

We conducted experiments for SSDA-MMD (ours), UDA-MMD, and Source-only, each representing one of the three groups of methods described in Section 5.5. The prediction performance was reported on the test set of the corresponding target domain dataset in each of the 12 domain adaptation scenarios with $NS = 1$. Other details of the training procedure were kept the same as described in Section 5.3.3, except that for each training unit of the target domain only sensor readings up to the pre-defined truncation level were used. Fig. 11 shows the prediction performance of these methods in terms of RMSE (a) and Score (b) as a function of the truncation level. Here, to eliminate the influence of the difference in the number of test target units in each scenario, the average score across the test units was reported instead of the sum, cf. Eq. (27). In general, it can be seen that SSDA-MMD tends to perform better when more data up to higher degradation levels, i.e. lower truncation levels, is available. In contrast, this correlation was not always observed in the case of UDA-MMD. Moreover, it is obvious that SSDA-MMD outperforms UDA-MMD in

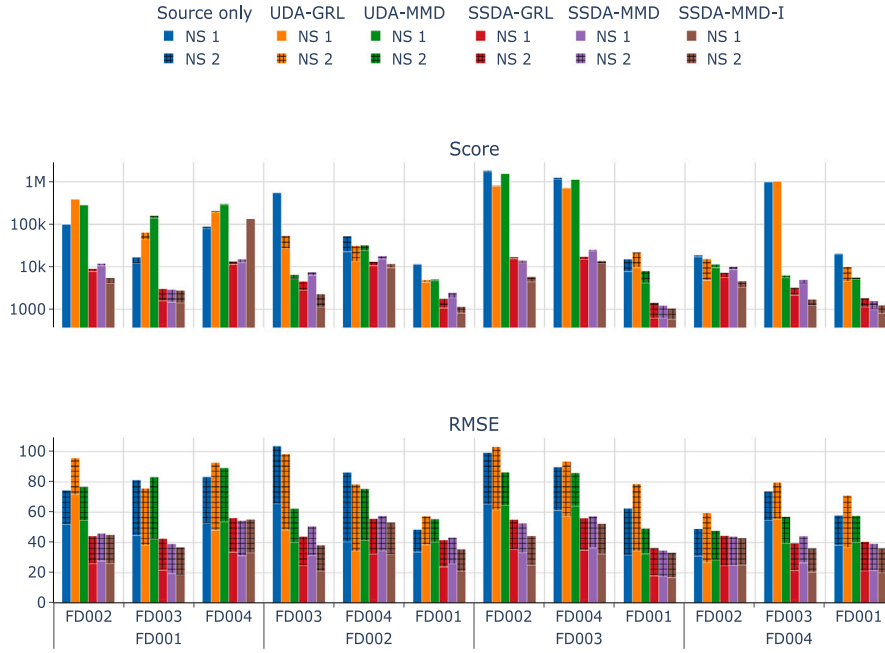


Fig. 10. Experimental results of the ablation studies (cf. Table 3).

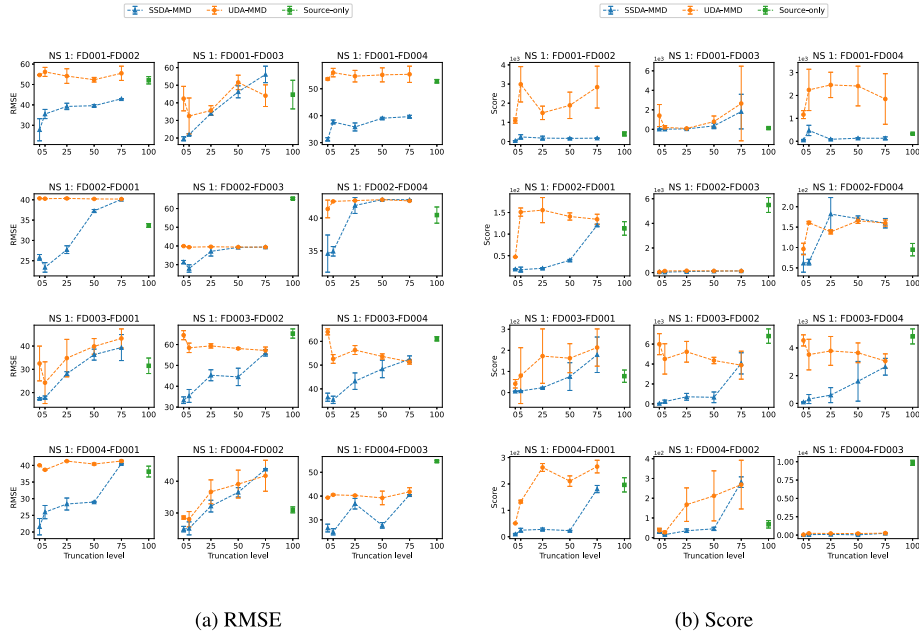


Fig. 11. Impact of target data availability on prediction performance.

most of the experimental scenarios, except for several scenarios with a truncation level of 75%, where the performance of these methods tends to be comparable. These observations once again confirm the benefit of the additional self-supervised semantic alignment in our proposed SSDA framework, especially when enough degradation data near failure in the target domain is available for training. Compared to Source-only, UDA-MMD often shows worse performance even at low truncation levels (e.g. for FD002 $\rightarrow$ FD001, FD004 $\rightarrow$ FD001), indicating the negative transfer of the domain adaptation when aligning only the marginal feature distributions. Our SSDA-MMD, on the other hand, shows superior performance compared to Source-only in most of the experimental scenarios, except for several scenarios with high truncation levels, e.g. 75% in FD001 $\rightarrow$ FD003, or 50% and 75% in FD002 $\rightarrow$ FD001. The negative transfer effect (domain adaptation

performs worse than Source-only) observed at high truncation levels despite the semantic alignment in SSDA-MMD can be explained by the fact that at high truncation levels, most of the sensor readings available for training are the data recorded when the engines were still in their healthy state. In these cases, the proposed additional self-supervision cannot utilize the available data of the target domain for the semantic alignment because there is too little or no degradation in the data.

6. Conclusions

In this work we presented a novel self-supervised domain adaptation (SSDA) framework for the task of remaining useful life (RUL) prediction. Data-driven models that are trained in a conventional supervised manner typically show poor performance in practice due to

the domain gap between the training and test data caused by the diversity in the working conditions and fault modes. Classical unsupervised domain adaptation (UDA) methods mitigate the domain shift problem by aligning the marginal feature distributions of the source and target domains, aiming to learn domain-invariant features while forcing the prediction error on the source labeled data to be low. Our empirical observations, however, showed that the domain-invariant feature representation and low prediction errors on the source domain data are insufficient to guarantee good generalization on the target domain data. To overcome these limitations, building upon the basis of the UDA methods, our proposed SSDA additionally incorporates the intrinsic characteristics of the target domain data into the domain adaptation process in a self-supervised manner using a novel dual Siamese network-based training pipeline. The self-supervised pre-text task is specifically designed for the RUL prediction task, capturing the nature of the underlying degradation process. Experimental results of extensive evaluation settings on the benchmark C-MAPSS dataset showed the superior performance of our proposed framework over the other state-of-the-art approaches, with an average improvement of 20.1% and 51.2% in terms of two different performance metrics. The ablation studies and the performance analyses under different degrees of target data availability provided further empirical evidence for the effectiveness of our proposed framework.

Despite promising experimental results, the proposed framework has some potential limitations worth discussing. Being aware of these limitations also opens up further research directions for future work. First, the self-supervised pre-text task proposed in this work does not cover scenarios where in the target domain only data from the healthy phase is available for training. This led to the negative transfer effect observed in the experimental scenarios with high truncation levels of the target domain data, as reported in Section 5.6. In future work, it will be of interest to take this data scenario into account when designing the pre-text task to address the aforementioned limitation. Second, although C-MAPSS is one of the most popular benchmark datasets for RUL prediction, it includes only simulated data, which may not contain enough variability often observed in real-world systems. Therefore, in future work, it will be worthwhile to evaluate the effectiveness of the proposed framework on further datasets, especially those from real-world systems.

CRedit authorship contribution statement

Quy Le Xuan: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Conceptualization. **Marco Munderloh:** Writing – review & editing, Validation, Project administration, Funding acquisition, Conceptualization. **Jörn Ostermann:** Writing – review & editing, Supervision, Resources, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by the Federal Ministry for Economic Affairs and Climate Action (BMWK), Germany, within the framework of the project IIP-Ecosphere (grant number 01MK20006A/D).

References

- [1] Zio E. Prognostics and Health Management (PHM): Where are we and where do we (need to) go in theory and practice. *Reliab Eng Syst Saf* 2022;218:108119. <http://dx.doi.org/10.1016/j.res.2021.108119>.

- [2] Lei Y, Li N, Gontarz S, Lin J, Radkowski S, Dybala J. A model-based method for remaining useful life prediction of machinery. *IEEE Trans Reliab* 2016;65(3):1314–26. <http://dx.doi.org/10.1109/TR.2016.2570568>.
- [3] Cubillo A, Perinpanayagam S, Esperon-Giguez M. A review of physics-based models in prognostics: Application to gears and bearings of rotating machinery. *Adv Mech Eng* 2016;8(8). <http://dx.doi.org/10.1177/1687814016664660>.
- [4] Lei Y, Li N, Guo L, Li N, Yan T, Lin J. Machinery health prognostics: A systematic review from data acquisition to RUL prediction. *Mech Syst Signal Process* 2018;104:799–834. <http://dx.doi.org/10.1016/j.ymssp.2017.11.016>.
- [5] Serradilla O, Zugasti E, Rodriguez J, Zurutuza U. Deep learning models for predictive maintenance: a survey, comparison, challenges and prospects. *Appl Intell* 2022;52(10):10934–64. <http://dx.doi.org/10.1007/s10489-021-03004-y>.
- [6] Sateesh Babu G, Zhao P, Li X-L. Deep convolutional neural network based regression approach for estimation of remaining useful life. In: Database systems for advanced applications: 21st international conference, DASFAA 2016, Dallas, TX, USA, April 16–19, 2016, proceedings, part i 21. Springer; 2016, p. 214–28. http://dx.doi.org/10.1007/978-3-319-32025-0_14.
- [7] Li X, Ding Q, Sun J-Q. Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliab Eng Syst Saf* 2018;172:1–11. <http://dx.doi.org/10.1016/j.res.2017.11.021>.
- [8] Zheng S, Ristovski K, Farahat A, Gupta C. Long short-term memory network for remaining useful life estimation. In: 2017 IEEE international conference on prognostics and health management. ICPHM, 2017, p. 88–95. <http://dx.doi.org/10.1109/ICPHM.2017.7998311>.
- [9] Shi J, Zhong J, Zhang Y, Xiao B, Xiao L, Zheng Y. A dual attention LSTM lightweight model based on exponential smoothing for remaining useful life prediction. *Reliab Eng Syst Saf* 2023;243:109821. <http://dx.doi.org/10.1016/j.res.2023.109821>.
- [10] Ma M, Mao Z. Deep-convolution-based LSTM network for remaining useful life prediction. *IEEE Trans Ind Inf* 2020;17(3):1658–67. <http://dx.doi.org/10.1109/TII.2020.2991796>.
- [11] Ren L, Sun Y, Cui J, Zhang L. Bearing remaining useful life prediction based on deep autoencoder and deep neural networks. *J Manuf Syst* 2018;48:71–7. <http://dx.doi.org/10.1016/j.jmsy.2018.04.008>.
- [12] Yu W, Kim IY, Mechefske C. An improved similarity-based prognostic algorithm for RUL estimation using an RNN autoencoder scheme. *Reliab Eng Syst Saf* 2020;199:106926. <http://dx.doi.org/10.1016/j.res.2020.106926>.
- [13] Mo Y, Wu Q, Li X, Huang B. Remaining useful life estimation via transformer encoder enhanced by a gated convolutional unit. *J Intell Manuf* 2021;32:1997–2006. <http://dx.doi.org/10.1007/s10845-021-01750-x>.
- [14] Liu L, Song X, Zhou Z. Aircraft engine remaining useful life estimation via a double attention-based data-driven architecture. *Reliab Eng Syst Saf* 2022;221:108330. <http://dx.doi.org/10.1016/j.res.2022.108330>.
- [15] Yao S, Kang Q, Zhou M, Rawa MJ, Abusorrah A. A survey of transfer learning for machinery diagnostics and prognostics. *Artif Intell Rev* 2022. <http://dx.doi.org/10.1007/s10462-022-10230-4>.
- [16] Chen J, Huang R, Chen Z, Mao W, Li W. Transfer learning algorithms for bearing remaining useful life prediction: A comprehensive review from an industrial application perspective. *Mech Syst Signal Process* 2023;193:110239. <http://dx.doi.org/10.1016/j.ymssp.2023.110239>.
- [17] Kou W, Loog M. A review of domain adaptation without target labels. *IEEE Trans Pattern Anal Mach Intell* 2021;43(3):766–85. <http://dx.doi.org/10.1109/TPAMI.2019.2945942>, Conference Name: IEEE Transactions on Pattern Analysis and Machine Intelligence.
- [18] Tzeng E, Hoffman J, Zhang N, Saenko K, Darrell T. Deep domain confusion: Maximizing for domain invariance. 2014, arXiv preprint [arXiv:1412.3474](https://arxiv.org/abs/1412.3474).
- [19] Long M, Cao Y, Wang J, Jordan M. Learning transferable features with deep adaptation networks. In: International conference on machine learning. PMLR; 2015, p. 97–105.
- [20] Ganin Y, Lempitsky V. Unsupervised domain adaptation by backpropagation. In: International conference on machine learning. PMLR; 2015, p. 1180–9.
- [21] Ganin Y, Ustinova E, Ajakan H, Germain P, Larochelle H, Laviolette F, Marchand M, Lempitsky V. Domain-adversarial training of neural networks. *J Mach Learn Res* 2016;17(59):1–35.
- [22] Sun B, Feng J, Saenko K. Return of frustratingly easy domain adaptation. In: Proceedings of the AAAI conference on artificial intelligence. vol. 30, 2016.
- [23] Tzeng E, Hoffman J, Saenko K, Darrell T. Adversarial discriminative domain adaptation. In: 2017 IEEE conference on computer vision and pattern recognition. CVPR, Honolulu, HI: IEEE; 2017, p. 2962–71. <http://dx.doi.org/10.1109/CVPR.2017.316>.
- [24] Long M, Zhu H, Wang J, Jordan M. Unsupervised domain adaptation with residual transfer networks. *Adv Neural Inf Process Syst* 2016;29.
- [25] Long M, Zhu H, Wang J, Jordan M. Deep transfer learning with joint adaptation networks. In: International conference on machine learning. PMLR; 2017, p. 2208–17.
- [26] Bousmalis K, Silberman N, Dohan D, Erhan D, Krishnan D. Unsupervised pixel-level domain adaptation with generative adversarial networks. In: 2017 IEEE conference on computer vision and pattern recognition. CVPR, Honolulu, HI: IEEE; 2017, p. 95–104. <http://dx.doi.org/10.1109/CVPR.2017.18>.

- [27] Hoffman J, Tzeng E, Park T, Zhu J-Y, Isola P, Saenko K, Efros A, Darrell T. Cycada: Cycle-consistent adversarial domain adaptation. In: *International conference on machine learning*. Pmlr; 2018, p. 1989–98.
- [28] Vogt K, Paul A, Ostermann J, Rottensteiner F, Heipke C. Unsupervised source selection for domain adaptation. *Photogramm Eng Remote Sens* 2018;84(5):249–61. <http://dx.doi.org/10.14358/PERS.84.5.249>.
- [29] Kuhnke F, Ostermann J. Deep head pose estimation using synthetic images and partial adversarial domain adaption for continuous label spaces. In: *2019 IEEE/CVF international conference on computer vision. ICCV, Seoul, Korea (South): IEEE*; 2019, p. 10163–72. <http://dx.doi.org/10.1109/ICCV.2019.01026>.
- [30] Li W, Huang R, Li J, Liao Y, Chen Z, He G, Yan R, Gryllias K. A perspective survey on deep transfer learning for fault diagnosis in industrial scenarios: Theories, applications and challenges. *Mech Syst Signal Process* 2022;167:108487. <http://dx.doi.org/10.1016/j.ymssp.2021.108487>.
- [31] Shen C, Wang X, Wang D, Li Y, Zhu J, Gong M. Dynamic joint distribution alignment network for bearing fault diagnosis under variable working conditions. *IEEE Trans Instrum Meas* 2021;70:1–13. <http://dx.doi.org/10.1109/TIM.2021.3055786>.
- [32] da Costa PRD, Akçay A, Zhang Y, Kaymak U. Remaining useful lifetime prediction via deep domain adaptation. *Reliab Eng Syst Saf* 2020;195:106682. <http://dx.doi.org/10.1016/j.res.2019.106682>.
- [33] Ragab M, Chen Z, Wu M, Kwok CK, Li X. Adversarial transfer learning for machine remaining useful life prediction. In: *2020 IEEE international conference on prognostics and health management. ICPHM, 2020*, p. 1–7. <http://dx.doi.org/10.1109/ICPHM49022.2020.9187053>.
- [34] Fu S, Zhang Y, Lin L, Zhao M, Zhong S-s. Deep residual LSTM with domain-invariance for remaining useful life prediction across domains. *Reliab Eng Syst Saf* 2021;216:108012. <http://dx.doi.org/10.1016/j.res.2021.108012>.
- [35] Ragab M, Chen Z, Wu M, Foo CS, Kwok CK, Yan R, Li X. Contrastive adversarial domain adaptation for machine remaining useful life prediction. *IEEE Trans Ind Inf* 2021;17(8):5239–49. <http://dx.doi.org/10.1109/TII.2020.3032690>.
- [36] Zhang W, Li X, Ma H, Luo Z, Li X. Transfer learning using deep representation regularization in remaining useful life prediction across operating conditions. *Reliab Eng Syst Saf* 2021;211:107556. <http://dx.doi.org/10.1016/j.res.2021.107556>.
- [37] Hu T, Guo Y, Gu L, Zhou Y, Zhang Z, Zhou Z. Remaining useful life estimation of bearings under different working conditions via Wasserstein distance-based weighted domain adaptation. *Reliab Eng Syst Saf* 2022;224:108526. <http://dx.doi.org/10.1016/j.res.2022.108526>.
- [38] Li X, Xu Y, Li N, Yang B, Lei Y. Remaining useful life prediction with partial sensor malfunctions using deep adversarial networks. *IEEE/CAA J Autom Sin* 2023;10(1):121–34. <http://dx.doi.org/10.1109/JAS.2022.105935>.
- [39] Nejjar J, Geissmann F, Zhao M, Taal C, Fink O. Domain adaptation via alignment of operation profile for remaining useful lifetime prediction. *Reliab Eng Syst Saf* 2023;242:109718. <http://dx.doi.org/10.1016/j.res.2023.109718>.
- [40] Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y. Generative adversarial nets. In: Ghahramani Z, Welling M, Cortes C, Lawrence N, Weinberger K, editors. *Advances in neural information processing systems*. vol. 27, Curran Associates, Inc.; 2014.
- [41] Zhao H, Des Combes RT, Zhang K, Gordon G. On learning invariant representations for domain adaptation. In: *International conference on machine learning*. PMLR; 2019, p. 7523–32.
- [42] Oord Avd, Li Y, Vinyals O. Representation learning with contrastive predictive coding. 2018, arXiv preprint [arXiv:1807.03748](https://arxiv.org/abs/1807.03748).
- [43] Henaff O. Data-efficient image recognition with contrastive predictive coding. In: *International conference on machine learning*. PMLR; 2020, p. 4182–92.
- [44] Zhang Z, Chen X, Zio E, Li L. Multi-task learning boosted predictions of the remaining useful life of aero-engines under scenarios of working-condition shift. *Reliab Eng Syst Saf* 2023;237:109350. <http://dx.doi.org/10.1016/j.res.2023.109350>.
- [45] Ericsson L, Gouk H, Loy CC, Hospedales TM. Self-supervised representation learning: Introduction, advances, and challenges. *IEEE Signal Process Mag* 2022;39(3):42–62. <http://dx.doi.org/10.1109/MSP.2021.3134634>.
- [46] Kuhnke F, Ostermann J. Domain adaptation for head pose estimation using relative pose consistency. *IEEE Trans Biom Behav Identity Sci* 2023;5(3):348–59. <http://dx.doi.org/10.1109/TBIOM.2023.3237039>.
- [47] Guo H, Zhu H, Wang J, Vadakkepat P, Ho WK, Lee TH. Masked self-supervision for remaining useful lifetime prediction in machine tools. In: *2022 IEEE 20th international conference on industrial informatics. INDIN, Perth, Australia: IEEE*; 2022, p. 353–8. <http://dx.doi.org/10.1109/INDIN51773.2022.9976158>.
- [48] Akrim A, Gogu C, Vingerhoeds R, Salain M. Self-supervised learning for data scarcity in a fatigue damage prognostic problem. *Eng Appl Artif Intell* 2023;120:105837. <http://dx.doi.org/10.1016/j.engappai.2023.105837>.
- [49] Ding Y, Jia M, Cao Y, Ding P, Zhao X, Lee C-G. Domain generalization via adversarial out-domain augmentation for remaining useful life prediction of bearings under unseen conditions. *Knowl-Based Syst* 2023;261:110199. <http://dx.doi.org/10.1016/j.knsys.2022.110199>.
- [50] Bromley J, Guyon I, LeCun Y, Säckinger E, Shah R. Signature verification using a "siamese" time delay neural network. *Adv Neural Inf Process Syst* 1993;6.
- [51] Krokotsch T, Knaak M, Gühmann C. Improving semi-supervised learning for remaining useful lifetime estimation through self-supervision. *Int J Progn Health Manag* 2022;13. <http://dx.doi.org/10.36001/ijphm.2022.v13i1.3096>.
- [52] Vermelin WS, Löfberg A, Kyprianidis K. Self-supervised learning for efficient remaining useful life prediction. In: *Annual conference of the PHM society*. vol. 14, 2022, <http://dx.doi.org/10.36001/phmconf.2022.v14i1.3222>.
- [53] Ben-David S, Blitzer J, Crammer K, Kulesza A, Pereira F, Vaughan JW. A theory of learning from different domains. *Mach Learn* 2010;79(1–2):151–75. <http://dx.doi.org/10.1007/s10994-009-5152-4>.
- [54] Cortes C, Mohri M. Domain adaptation in regression. In: *Algorithmic learning theory: 22nd international conference, ALT 2011, espoo, Finland, October 5–7, 2011. proceedings 22*. Springer; 2011, p. 308–23.
- [55] Farahani A, Voghoei S, Rasheed K, Arabnia HR. A brief review of domain adaptation. In: *Advances in data science and information engineering: Proceedings from ICDATA 2020 and IKE 2020*. Springer; 2021, p. 877–94.
- [56] Mehra A, Kailkhura B, Chen P-Y, Hamm J. Understanding the limits of unsupervised domain adaptation via data poisoning. *Adv Neural Inf Process Syst* 2021;34:17347–59.
- [57] Johansson FD, Sontag D, Ranganath R. Support and invertibility in domain-invariant representations. In: *The 22nd international conference on artificial intelligence and statistics*. PMLR; 2019, p. 527–36.
- [58] Gretton A, Borgwardt KM, Rasch M, Schölkopf B, Smola AJ. A kernel method for the two-sample-problem. In: *Schölkopf B, Platt J, Hofmann T, editors. Advances in neural information processing systems 19*. The MIT Press; 2007, p. 513–20. <http://dx.doi.org/10.7551/mitpress/7503.003.0069>.
- [59] Gretton A, Borgwardt KM, Rasch MJ, Schölkopf B, Smola A. A kernel two-sample test. *J Mach Learn Res* 2012;13(1):723–73.
- [60] Pan SJ, Tsang IW, Kwok JT, Yang Q. Domain adaptation via transfer component analysis. *IEEE Trans Neural Netw* 2011;22(2):199–210. <http://dx.doi.org/10.1109/TNN.2010.2091281>.
- [61] Li Y, Swersky K, Zemel R. Generative moment matching networks. In: *International conference on machine learning*. PMLR; 2015, p. 1718–27.
- [62] Gretton A, Sejdinovic D, Strathmann H, Balakrishnan S, Pontil M, Fukumizu K, Sriperumbudur BK. Optimal kernel choice for large-scale two-sample tests. *Adv Neural Inf Process Syst* 2012;25.
- [63] Saxena A, Goebel K. Turbofan engine degradation simulation data set. *NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA*; 2008.
- [64] Saxena A, Goebel K, Simon D, Eklund N. Damage propagation modeling for aircraft engine run-to-failure simulation. In: *2008 international conference on prognostics and health management*. 2008, p. 1–9. <http://dx.doi.org/10.1109/PHM.2008.4711414>.
- [65] McInnes L, Healy J, Melville J. Umap: Uniform manifold approximation and projection for dimension reduction. 2018, arXiv preprint [arXiv:1802.03426](https://arxiv.org/abs/1802.03426).
- [66] Li J, Li X, He D. Domain adaptation remaining useful life prediction method based on AdaBN-DCNN. In: *2019 prognostics and system health management conference. PHM-Qingdao, 2019*, p. 1–6. <http://dx.doi.org/10.1109/PHM-Qingdao46334.2019.8942857>.
- [67] Akiba T, Sano S, Yanase T, Ohta T, Koyama M. Optuna: A next-generation hyperparameter optimization framework. In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. Anchorage AK USA: ACM; 2019, p. 2623–31. <http://dx.doi.org/10.1145/3292500.3330701>.
- [68] Tzeng E, Hoffman J, Darrell T, Saenko K. Simultaneous deep transfer across domains and tasks. In: *Proceedings of the IEEE international conference on computer vision*. 2015, p. 4068–76.